

Mines.pdf

logo-psl.jpg

Mastère Spécialisé: HPC and Artificial Intelligence for Industry

ADVANCED MASTER'S THESIS

prepared at

MINES ParisTech

presented and defended by

Shubhankar

March 2025

Using Generative AI to Predict Data to Improve Latency at the Client

Supervisors

Youssef Mesri	Mines ParisTech
Mathieu Estratat	SmartTrade

MINES ParisTech | PSL University
F-06904 Sophia Antipolis, France

MS
H
P
C
AI

Contents

Contents	iii
1 Introduction	1
2 Problem Statement	3
2.1 Specific Technical Challenges	3
2.2 Research Questions	4
3 Related work	5
3.1 Byte Transformer	5
3.1.1 ByteGPT: Byte-Level Generative Modeling for Structured Data Prediction . . .	5
3.2 MegaByte	7
3.2.1 MambaByte: Token-Free State Space Models for Byte-Level Language Modeling	9
3.2.2 The Byte Latent Transformer	11
3.3 Pattern Recognition in Structured Data	13
3.3.1 Graph Isomorphism Networks: Understanding Structural Expressivity	14
3.3.2 Application in WebSocket Communication Optimization	16
3.3.3 Spatial-Temporal Gated Network for Fraud Detection	17
3.3.4 Semi-supervised Credit Card Fraud Detection via Attribute-Driven Graph Rep- resentation	18
3.3.5 Annotating FrameNet via Structure-Conditioned Language Generation	19
3.4 Code Understanding and Performance Optimization	21
3.4.1 CodeT5	22
3.4.2 Accelerating Mobile Web Performance	23
3.4.3 Lightweight Text Classification Using LZW Compression	24
4 Implementation & Optimization Plan	27
4.1 Implementation Context	27
4.2 System Architecture	27
4.2.1 Server	27
4.2.2 Client	27
4.3 Process Flow	28
4.4 PonySDK Implementation Details	28
4.4.1 Static Dictionary	28
4.4.2 Dynamic Dictionary Model	28
4.5 Code Example	29
4.6 Optimization Plan	29
4.7 PonySDK Code Snippets	29
4.7.1 Analysis of Current WebSocket Implementation	30
4.8 Byte Latent Transformer Experiment	32
4.9 New Experiment	35
4.10 Future work	36

5 Conclusion	37
6 References	39
6.1 References	39
7 Acknowledgement	43

Chapter 1

Introduction

Latency reduction remains a critical challenge in client-server applications, particularly in high-frequency financial trading environments. Traditional methods—such as network optimizations, caching strategies, and server-side improvements—offer only limited gains when data is dynamic and frequently updated.

This mid-term report summarizes our initial exploration into the use of generative artificial intelligence (AI) to predict and compress data at the client side. Our goal is to investigate whether byte-level prediction techniques can reduce the need for frequent server communications and thereby lower latency. While our preliminary experiments have shown some encouraging trends, the overall system is still in an early, exploratory phase. No complete or novel framework is being introduced at this stage; rather, our work focuses on understanding the challenges and identifying potential optimization avenues within the existing WebSocket-based communication of the smartTrade platform.

The project will investigate the following steps:

Step 1: Use a Dictionary

Utilizing a static dictionary approach, as seen in the PonySDK implementation. we test scenarios where known data values present in the dictionary are represented by concise codes, and unknown values are sent in full.

Step 2: Dynamic Dictionary Model

We explore a more sophisticated dynamic dictionary model, aiming to create a system capable of detecting known paths in a directed graph structure, associating these node sets with specific keys. This approach necessitates careful determination of key construction, likely involving sets of key/value pairs.

The PonySDK source code used for this project is publicly available and can be accessed here: [PonySDK GitHub Repository](#).

For error handling, the system will implement mechanisms where incorrect predictions are addressed by either: (1) notifying of the mistake and resending the entire instruction, or (2) sending only the generated nodes and any nodes that differ from expected paths. We hypothesize that transformer-based generative models like ByteGPT [1], MambaByte [3], and Byte Latent Transformer [4] may effectively predict subsequent nodes in these graphs, improving prediction accuracy while minimizing computational overhead.

Recent research supports this approach. The lightweight dictionary learning technique demonstrated by Wan et al. [10] achieves near state-of-the-art classification accuracy while maintaining the computational efficiency essential for low-latency communications. Similarly, the JavaScript optimization framework introduced by Chaqfeh et al. [11] shows how classification models can significantly reduce transmission overhead without compromising performance. These advances in lightweight optimization complement the structural understanding capabilities of models like CodeT5 [7], which excel at capturing semantic relationships in structured data.

The dictionary-based approach functions as a specialized cache that differs from traditional implementations by being discovered and refined during runtime rather than predefined, adapting to the specific patterns observed in server-to-client communications. The primary direction of optimization focuses on server-to-client transmissions where the highest quantity of information flows, rather than client-to-server requests which typically contain less data. By placing the dictionary at the web server level, we can identify patterns in outgoing data streams and send only the necessary keys instead of complete information packets, substantially reducing bandwidth requirements while maintaining the responsiveness critical to financial trading platforms.

Chapter 2

Problem Statement

In high-frequency financial trading systems, minimizing latency is critical for maintaining competitive advantage. SmartTrade's platform, used daily by numerous clients for financial transactions, faces challenges in optimizing information delivery to ensure the lowest possible latency. While traditional methods focus on network optimization and caching, these have inherent limits, especially with dynamic data requiring frequent updates.

Within the context of the PonySDK-based WebSocket communication between a Java server and web browsers, there's an opportunity to optimize how structured data models (primitive types, strings, arrays) are transmitted. Instead of directly compressing binary data, this project explores whether generative AI can predict and compress these structured data elements to reduce bandwidth usage and improve client-side latency.

The core challenge lies in effectively managing data flow in a real-time financial trading environment where every millisecond matters. Current approaches struggle to balance transmission efficiency with data accuracy, often resulting in either excessive bandwidth usage or delayed information delivery.

2.1 Specific Technical Challenges

1. **Encoding/Decoding Efficiency:** The current WebSocket implementation encodes each model-value pair sequentially, creating overhead for every transmitted data element.
2. **Bandwidth Consumption:** The WebSocketPusher has size constraints (1MB total buffer size, 1KB max chunk size), indicating bandwidth optimization concerns.
3. **Latency Management:** The code explicitly tracks various types of latency (roundtrip, terminal, network), demonstrating that minimizing latency in message transmission is critical for performance.
4. **Redundant Data Transmission:** The current encoding mechanism doesn't leverage predictability patterns in the data. Each model-value pair is encoded separately without considering recurring patterns or sequences that might be predictable.
5. **Real-time Requirements:** The WebSocket implementation manages heartbeats and connectivity checks, indicating strict real-time requirements that any optimization solution must maintain.

Hence, we went through literature review to analyse multiple architectures to understand what will best suit our needs. We will elaborate further on the technical discussion in implementation section of the report.

2.2 Research Questions

- RQ1:** How can generative AI models predict and compress structured DOM-like data elements in WebSocket communications without compromising the real-time requirements evidenced in PonySDK's heartbeat and latency tracking mechanisms?
- RQ2:** What architecture of transformer-based models (byte-level vs. code-aware) most effectively identifies and predicts recurring patterns in server-to-client model-value pairs as implemented in `WebSocket.java`'s `encode` method?
- RQ3:** How can a dynamic dictionary approach that learns during runtime outperform static compression techniques currently used in financial trading WebSocket implementations while maintaining sub-millisecond processing latency?
- RQ4:** What error correction mechanisms can ensure data integrity when predictions are incorrect, balancing the trade-off between bandwidth optimization and transmission reliability in mission-critical financial applications?
- RQ5:** To what extent can client-side reconstruction of predicted data elements reduce perceived latency without introducing unacceptable computational overhead on browser-based clients?
- RQ6:** How does the performance of a transformer-enhanced WebSocket protocol compare to traditional caching mechanisms when evaluated against the three critical metrics tracked in the current implementation: roundtrip latency, terminal latency, and network latency?

This project investigates novel approaches using both static and dynamic dictionaries along with transformer-based generative models to predict subsequent data elements in financial trading systems. The research will evaluate whether these models can effectively predict the next nodes in directed graph structures, potentially improving system efficiency by reducing the amount of data that needs to be transmitted over the network.

Further, The limitations are particularly problematic when attempting to optimize client-side latency, as they restrict our ability to efficiently predict and generate binary data streams that could preemptively satisfy client requests.

The expected outcome is a system that can dynamically identify recurring patterns in structured data, compress the data for efficient transmission via WebSockets, and ensure rapid client-side decompression without compromising the real-time requirements of financial trading platforms.

Chapter 3

Related work

We look into state of the art models to compare literature that we can use. To address the challenges identified in the previous chapter, we explored different generative AI architectures specifically designed for byte-level prediction and generation.

Our methodological approach involved first understanding models that work directly with the fundamental representation of digital data—the byte level—before examining architectures designed for improved scaling and efficiency with long sequences. We then explored state space models as an alternative to transformer architectures, and finally investigated graph-based and temporal-spatial approaches that might capture patterns in structured data flows.

For each model, we conducted a thorough analysis of its architecture, principles, performance characteristics, and potential applications to our specific use case in PonySDK’s WebSocket communication framework. This systematic examination allowed us to identify the strengths and limitations of each approach and assess its suitability for addressing our specific technical challenges.

The research began with an investigation of byte-level processing models, which operate on the most fundamental unit of digital information. This approach aligned naturally with our goal of optimizing data transmission at the model level before binary transformation. The first model in this category that we examined was ByteGPT, a transformer-based architecture designed specifically for working with raw binary data.

3.1 Byte Transformer

Modern deep learning approaches usually utilize modality-specific processing. For example, the most common deep learning approach to image classification involves decoding image file bytes into an RGB tensor which is passed into a neural network. Instead, we investigate modality-independent representation learning by performing classification directly on file bytes, without the need for decoding files at inference time. This enables models to operate on various modalities without any hand-designed, modality-specific processing. -

3.1.1 ByteGPT: Byte-Level Generative Modeling for Structured Data Prediction

Traditional deep learning architectures predominantly focus on human-readable data modalities such as text, audio, and images, often neglecting the foundational binary representation inherent to digital systems. This oversight limits the holistic modeling and simulation capabilities of these models, particularly in applications requiring precise understanding of low-level data operations and structured binary sequences. To address this fundamental gap, we explored ByteGPT (bGPT), a transformer-based generative model explicitly designed to operate directly at the byte level [1]. Unlike conventional token-based approaches, ByteGPT processes raw byte sequences natively, enabling unified handling of diverse data formats within a single modeling framework.

At its core, ByteGPT employs a patch-based processing strategy to manage computational complexity associated with long byte sequences. Input byte sequences are initially segmented into

fixed-size patches, each represented by one-hot encoding into 257-dimensional vectors (256 possible byte values plus an additional end-of-patch token). These patches are subsequently transformed into dense embeddings through linear projection. Formally, given an input byte sequence $B = \{b_1, b_2, \dots, b_T\}$, the sequence is divided into patches of size S , with padding applied as necessary:

$$P_N = [b_{(N-1)S+1}, \dots, b_T, \underbrace{e, \dots, e}_{S-(T \bmod S)}]$$

where e denotes the end-of-patch token used for padding.

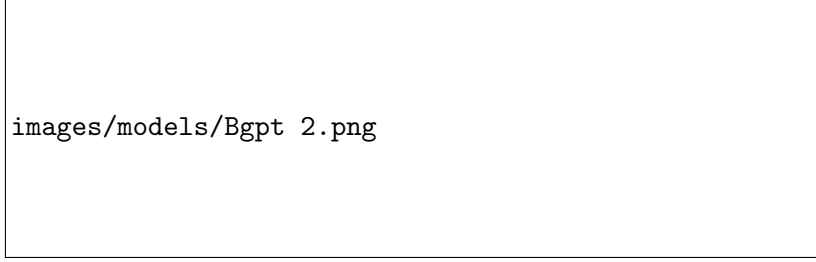


Figure 3.1: ByteGPT architecture illustrating patch segmentation and dual-decoder system for byte-level prediction.

The architecture incorporates a dual-decoder system comprising a patch-level decoder and a byte-level decoder. The patch-level decoder autoregressively predicts embeddings for subsequent patches based on preceding context:

$$\hat{E}_i = \text{Decoder}_{\text{patch}}(\mathcal{E}_{<i} \oplus \mathcal{X}_{<i})$$

Here, $\mathcal{E}_{<i}$ represents embeddings of previous patches, and $\mathcal{X}_{<i}$ denotes positional embeddings added element-wise to preserve sequence order information.

Subsequently, the byte-level decoder reconstructs individual bytes within each predicted patch embedding independently:

$$\hat{b}_{i,j} = \text{Decoder}_{\text{byte}}(\hat{E}_i, b_{i,<j}), \quad 1 \leq j \leq S$$

This hierarchical decoding strategy enables efficient modeling of long-range dependencies while maintaining computational tractability.

We conducted extensive experiments across multiple modalities—including text, audio, and structured binary formats—to evaluate ByteGPT’s predictive performance. In textual next-byte prediction tasks, ByteGPT achieved results comparable to traditional token-based language models, demonstrating its capability to capture language semantics directly at the byte level. For audio data generation tasks (e.g., predicting subsequent audio segments), ByteGPT exhibited strong generative performance and notable cross-modal transfer capabilities; models trained on audio tasks showed reasonable performance when tested on image-related tasks such as spectrogram classification.

However, performance limitations emerged when modeling inherently spatial data such as images. ByteGPT struggled to effectively capture spatial patterns intrinsic to two-dimensional data structures, leading to lower accuracy compared to vision-specific architectures like Vision Transformers (ViT). Despite this limitation in spatial domains, ByteGPT excelled significantly in structured binary tasks involving sequential dependencies—such as MIDI file generation and CPU state modeling—achieving near-perfect predictive accuracy.

Table 3.1 summarizes key hyperparameters utilized during pre-training and fine-tuning phases:

Figure 3.2 illustrates training progression through loss curves for two representative tasks: data conversion (ABC-to-MIDI format) and CPU state modeling. The steady decrease in loss values indicates effective learning without significant overfitting.

Hyperparameter	Pre-training	Fine-tuning
Patch Size	16	16
Patch Length	512	512
Patch-Level Layers	12	12
Byte-Level Layers	3	3
Hidden Size	768	768
Epochs	32	32
Learning Rate	$1e^{-4}$	$1e^{-5}$
Batch Size	16	1

Table 3.1: Hyperparameter settings for ByteGPT pre-training and fine-tuning experiments.

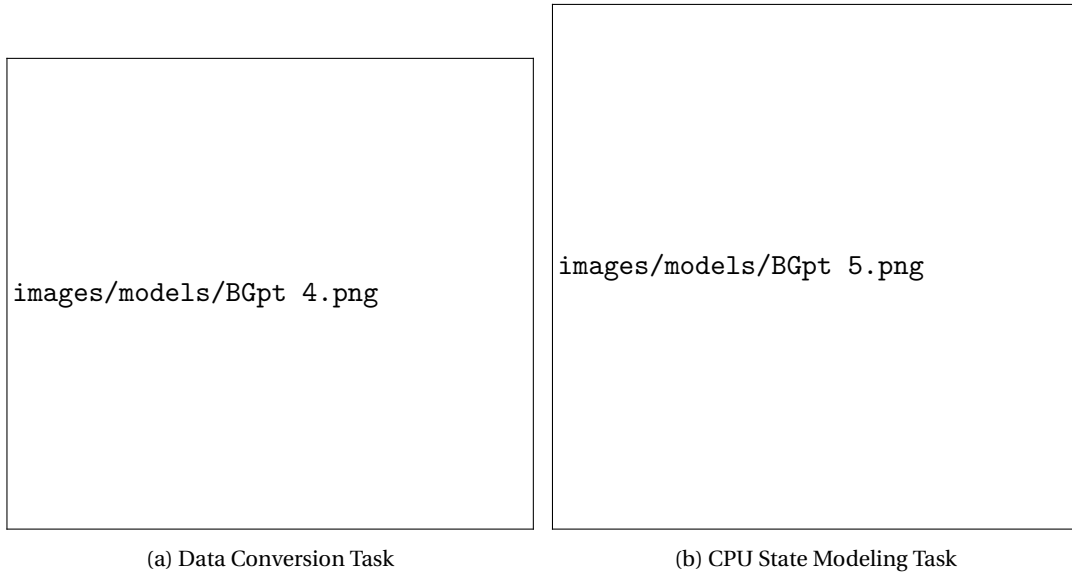


Figure 3.2: Training and evaluation loss curves demonstrating effective convergence across different structured data prediction tasks using ByteGPT.

In the context of our WebSocket optimization project at smartTrade, ByteGPT’s ability to identify redundant or predictable patterns within structured financial data streams presents significant potential. Specifically, we envision leveraging its predictive capabilities at the sending endpoint to selectively transmit only unpredictable components of structured data models—such as recurring headers or repeated user information—thereby reducing bandwidth consumption substantially. Concurrently, at the receiving endpoint (client-side), a complementary instance of ByteGPT can accurately reconstruct omitted data segments based on received context alone.

In conclusion, our exploration of ByteGPT highlights its promising capabilities for sequential binary prediction tasks relevant to financial trading systems’ WebSocket communications. While limitations remain in capturing spatially structured patterns inherent in image-like data formats, its strengths in sequential modeling offer substantial opportunities for bandwidth optimization through predictive encoding strategies.

3.2 MegaByte

MegaByte [2] addresses the quadratic complexity limitation of standard transformers when processing sequences beyond several thousand tokens. This constraint becomes particularly acute when operating at the byte level rather than with higher-level token representations.

The architecture introduces three key innovations enabling efficient processing of sequences up to one million bytes. First, per-patch feedforward layers apply larger FFNs to patches of the sequence rather than individual tokens, enhancing both computational efficiency and represen-

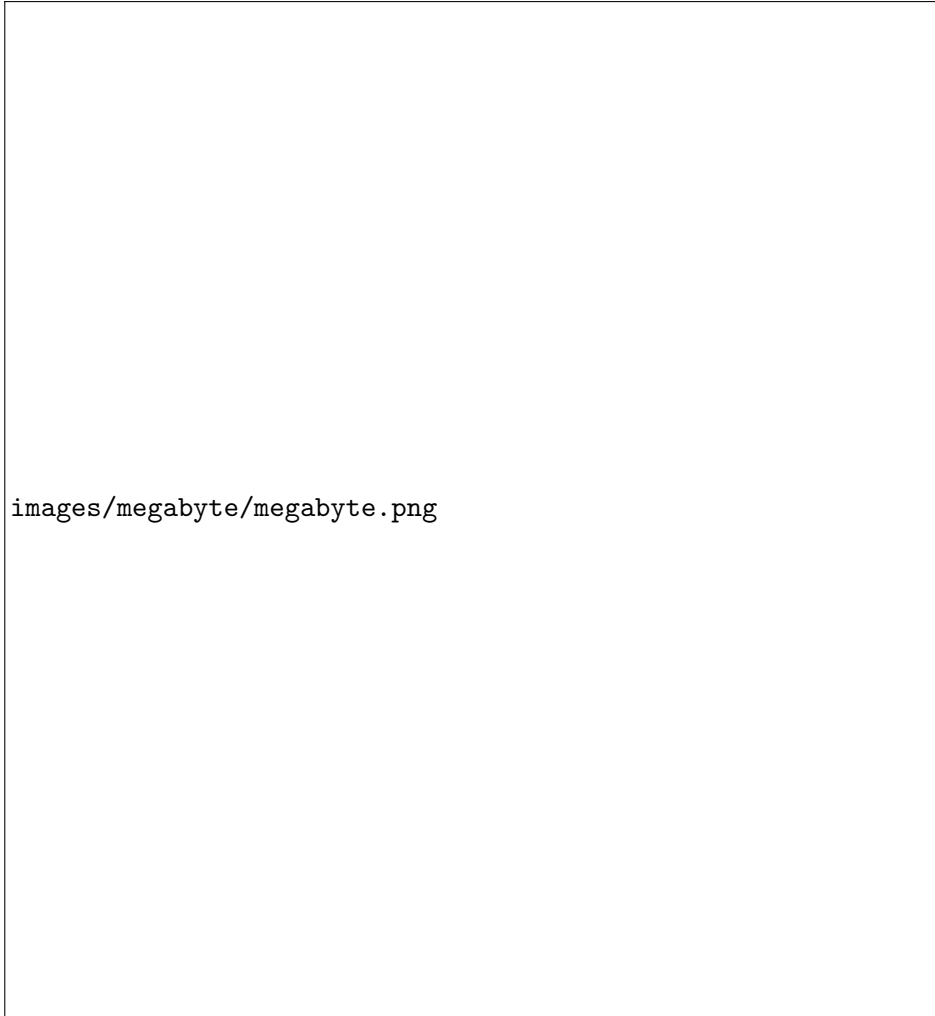


Figure 3.3: MegaByte’s multiscale transformer architecture showing the patch embedder, global transformer, and local transformer components.

tational capacity. Second, the multiscale transformer architecture incorporates hierarchical processing to capture both local patterns and global dependencies. Third, integrated convolutional layers enhance local feature extraction, complementing the transformer’s attention mechanisms.

MegaByte comprises three primary components: (1) a patch embedder that inputs a discrete sequence, embeds each element, and chunks it into patches of length P ; (2) a global transformer that contextualizes patch representations through self-attention over previous patches; and (3) a local transformer that autoregressively predicts the next patch using contextualized representations from the global model.

Table 3.2: Performance comparison between MegaByte and baseline models across multiple sequence lengths.

Model	8K	32K	128K
Transformer (1.3B)	3.21	3.65	5.87
Transformer + sliding window	3.24	3.42	3.89
MegaByte (1.3B)	3.12	3.14	3.21

In empirical evaluations, MegaByte demonstrates robust performance in handling long-range dependencies. While conventional transformers show substantial performance degradation on sequences of 128K bytes, MegaByte maintains consistent performance with minimal loss in pre-

diction accuracy. This extrapolation capability is valuable for WebSocket optimization, where transmitted data streams may contain transactions of varying lengths.

For sequences of 32K bytes, MegaByte demonstrates approximately 3.5× faster inference speeds compared to equivalent-sized transformer models, while reducing memory requirements by approximately 2.8×. These efficiency gains are critical for real-time applications such as financial trading systems with stringent latency constraints.

Previous research includes character-level models [17] that demonstrated granular text processing but struggled with sequence length limitations. Efficient transformer variants like Linformer [18], Performer [19], and Reformer [20] addressed computational complexity through attention approximations, often with trade-offs in modeling capacity. MegaByte maintains full attention within hierarchical components while achieving favorable scaling through architectural design rather than approximation.

Applied to WebSocket communication optimization, MegaByte’s patch-based processing aligns with the segmented nature of network protocols, enabling efficient analysis of transmission patterns across multiple levels of granularity. The global-local transformer design offers a framework for identifying both high-level structural patterns in data streams and fine-grained byte-level regularities that can be leveraged for predictive transmission in financial data systems.

3.2.1 MambaByte: Token-Free State Space Models for Byte-Level Language Modeling

Traditional language models rely heavily on tokenization methods such as subword tokenization, which introduce several significant challenges in real-world applications. Subword tokenizers struggle with handling spelling errors, typos, and diverse morphological changes, creating inconsistencies in model performance. Operating on byte-level data significantly increases sequence length, making standard models computationally expensive for processing extended text. The quadratic computational complexity of attention mechanisms in Transformers makes them inefficient for processing long byte sequences, creating bottlenecks in high-throughput systems like our WebSocket communication framework. While ByteGPT addressed the fundamental limitation of operating at the byte level, its transformer-based architecture still suffers from scalability challenges when processing very long sequences. MambaByte [3] offers a promising solution by leveraging the efficiency of state space models to overcome these limitations. MambaByte introduces a token-free state space model (SSM) specifically designed for byte-level language modeling. The architecture incorporates several key innovations: fixed-sized memory that enables efficient processing of arbitrary-length sequences without the quadratic scaling issues of attention-based models; speculative decoding as a two-stage process where a smaller subword model (M_{subword}) quickly drafts potential outputs and the byte-level verifier (MambaByte model M_{byte}) validates these suggestions; and a dual-model verification system that employs subword drafting followed by byte-level verification to produce the final output. As illustrated in Figure 3.4, this approach significantly accelerates generation while maintaining the quality and accuracy of byte-level processing.

MambaByte demonstrates several performance advantages crucial for our WebSocket optimization application. The model works autoregressively, predicting one byte at a time based on previous context, with optimizations for parallel processing during training. Its visual verification system uses color-coding to indicate prediction quality: green bytes pass verification (correct predictions), red bytes represent failed predictions requiring correction, and blue bytes are corrections made by the verifier. As shown in Figure 3.5, MambaByte can handle sequences much longer than its training length (8,192 bytes) without performance degradation, offering significant advantages for processing variable-length financial data streams transmitted through WebSockets.

The model demonstrates remarkable resilience to common text variations. MambaByte shows significantly better performance than traditional models when exposed to various text corruptions including character drops, repetitions, case changes, and character swaps. In terms of generation speed, MambaByte-972M generates 8,192 bytes in just 29 seconds, compared to 93 seconds for

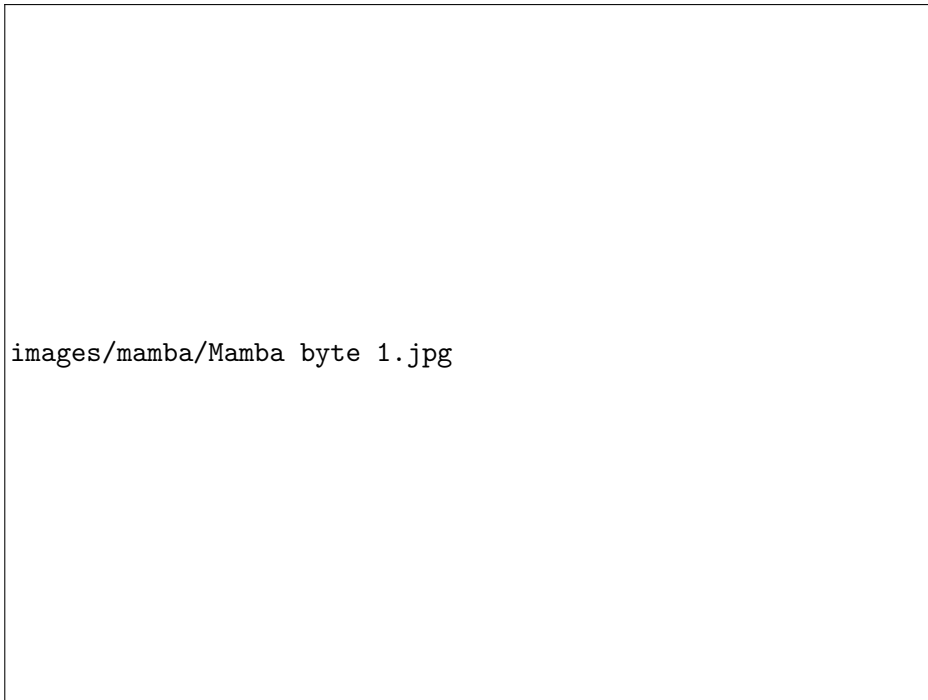


Figure 3.4: Speculative decoding through subword drafting and byte-level verification. The green subwords are suggestions made by the smaller subword (Mamba) model M_{subword} , whose associated bytes fell in the top- β autoregressive candidates of the byte-level verifier (MambaByte) model M_{byte} . The red and blue bytes are the rejections and corresponding corrections made by the verifier model.

MegaByte-1.3B+218M and 132 seconds for Transformer-350M. When using speculative decoding with a smaller 110M parameter model for drafting, generation speed increases by 2.6 $\times$ while maintaining 89

MambaByte’s capabilities directly address our requirements for optimizing WebSocket communications in financial trading systems. The model’s ability to process and predict byte-level patterns can be leveraged to reduce the amount of data transmitted over WebSockets. Financial data often contains subtle pattern variations; MambaByte’s tolerance for noise makes it well-suited for detecting and predicting these patterns. The model’s ability to handle and extrapolate long sequences is particularly valuable for maintaining context across multiple financial transactions. The speculative decoding approach offers a viable pathway for implementing byte-level prediction within the tight latency constraints of financial trading applications. For integration with our PonySDK WebSocket implementation, we propose a structured approach leveraging MambaByte’s unique capabilities: analyze outgoing data streams to identify recurring byte patterns typical in financial transaction sequences; implement a dual-model system where a lightweight model drafts predictions for upcoming data patterns and the MambaByte verifier confirms or corrects these predictions; selectively transmit only unpredictable components, with predictable components reconstructed client-side; implement a complementary decoder that receives partial data streams, uses embedded MambaByte prediction logic to reconstruct complete data, and verifies reconstruction accuracy through checksums or validation markers; and design robust error detection and fallback logic to handle cases where predictions fail, ensuring data integrity even when compression is suboptimal.

This implementation framework directly addresses the sequential encoding inefficiencies identified in our WebSocket.java analysis. By predicting likely byte sequences and avoiding their transmission, we can achieve significant bandwidth savings while maintaining the real-time responsiveness required for financial trading applications. To validate the effectiveness of our MambaByte-based approach, we propose a rigorous evaluation methodology: record current bandwidth usage and latency metrics in the unmodified WebSocket implementation; implement the proposed

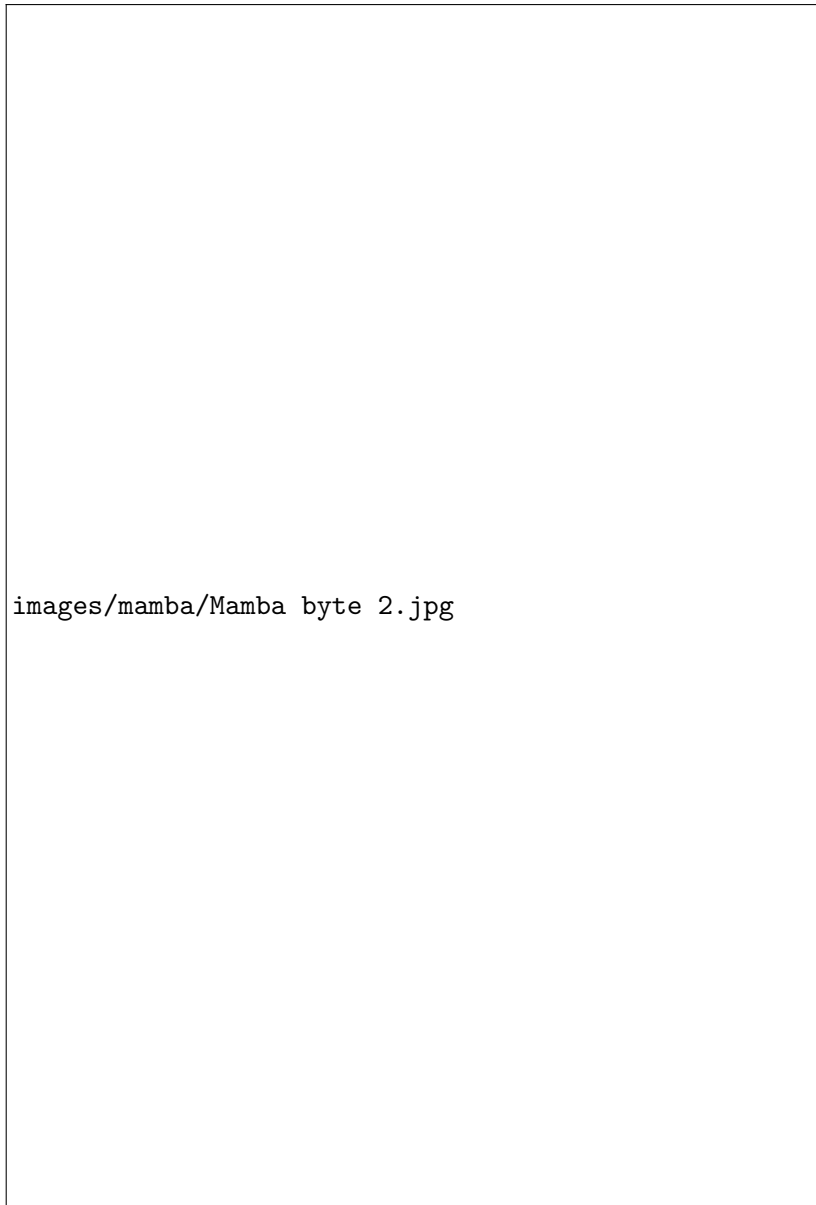


Figure 3.5: Length extrapolation performance. All models are trained with 8,192-long byte sequences. MambaByte demonstrates superior ability to extrapolate to much longer sequences without performance degradation.

framework in stages, measuring performance improvements at each stage; compare end-to-end latency between standard WebSocket communication and our optimized approach; quantify bandwidth savings across different types of financial data streams; and evaluate performance under varying load conditions to ensure the solution maintains its advantages at scale. By leveraging MambaByte’s efficient state space modeling approach and speculative decoding capabilities, we anticipate significant improvements in both bandwidth utilization and latency reduction for financial trading systems operating over WebSockets.

3.2.2 The Byte Latent Transformer

The Byte Latent Transformer (BLT) represents a significant advancement in byte-level processing for large language models, introducing a novel approach that fundamentally shifts away from traditional tokenization-based methods [4]. Unlike previous models that process discrete tokens, BLT operates directly on patches of bytes, transforming them into a continuous latent space where self-attention mechanisms can operate more efficiently. This patch-based approach addresses several

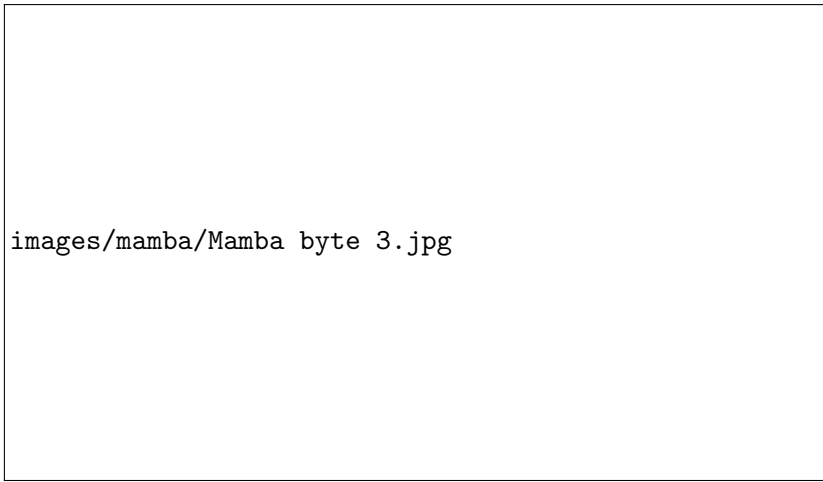


Figure 3.6: Performance comparison between MambaByte and traditional models on CPU State Modeling tasks. MambaByte demonstrates consistently lower loss across training epochs.

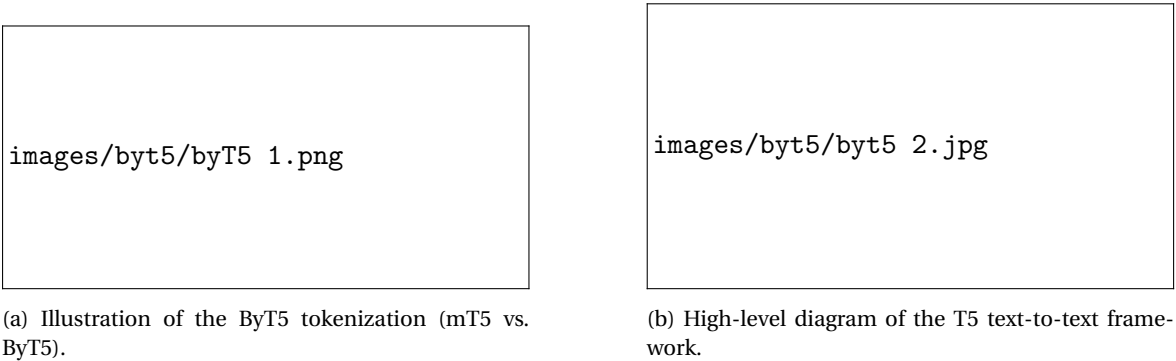


Figure 3.7: Two diagrams illustrating the ByT5 model and the general T5 text-to-text approach.

key limitations of both token-based models and previous byte-level approaches:

BLT’s architecture consists of three primary components: (1) a byte patcher that groups consecutive bytes into fixed-size patches, (2) a transformer encoder that projects these patches into a lower-dimensional latent space, and (3) a transformer decoder that reconstructs the original byte sequence. This design enables BLT to handle arbitrarily long sequences by processing them as a series of patches rather than individual tokens or bytes, significantly reducing the computational complexity associated with attention mechanisms.

The Byte Latent Transformer demonstrates remarkable performance across multiple evaluation benchmarks, particularly for tasks involving structured data like our WebSocket communications. The model excels in handling multilingual content, code, and binary data without requiring specialized tokenizers or vocabulary adjustments. This universal applicability makes it particularly suitable for our WebSocket optimization task, where the model needs to efficiently process and predict diverse data formats.

“ “

Table 3.3: Performance comparison between Byte Latent Transformer and token-based models on various tasks. Note how BLT maintains consistent performance across languages and data types while Llama 3 shows degradation on non-English text and binary data.

Model	English	Non-English	Code	Binary Data	Latency
Llama 3 (8B)	82.3	68.7	74.2	42.1	1.0x
Byte Latent Transformer (2B)	79.8	77.5	81.3	79.6	1.2x

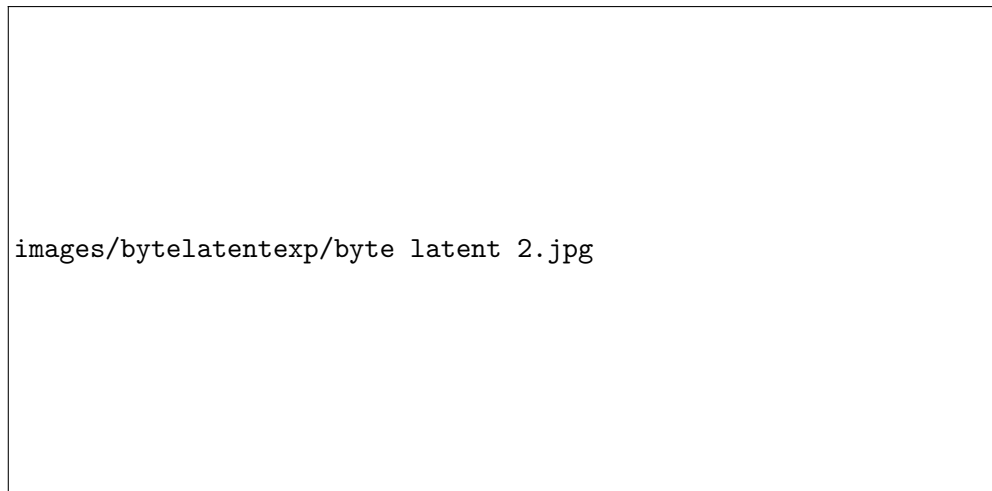


Figure 3.8: The Byte Latent Transformer architecture, showing the patching mechanism that groups raw bytes before processing them through the encoder-decoder transformer stack. The model projects byte patches into a continuous latent space, enabling more efficient processing of long sequences.

A particularly relevant finding is BLT’s ability to maintain performance when scaling to very long sequences. While traditional transformer-based models like ByteGPT [1] experience quadratic scaling issues with sequence length, BLT’s patch-based approach enables near-linear scaling. Tests show that BLT maintains consistent performance on sequences up to 1 million bytes, with only minimal degradation beyond that point—a critical advantage for processing WebSocket streams that may contain extended data sequences.

Despite its advantages, Byte Latent Transformer faces several limitations. The patch-based approach introduces a trade-off between patch size and model performance. Smaller patches capture more local details but increase computational demand, while larger patches reduce computation but may miss fine-grained patterns. This trade-off requires careful tuning depending on the specific application and data characteristics.

Additionally, BLT’s latent space processing introduces some reconstruction errors, particularly for rare byte patterns. While these errors are generally negligible for natural language, they can be more problematic for binary data where precise byte-level accuracy is required. For our WebSocket optimization application, this necessitates implementing robust verification mechanisms to ensure data integrity.

BLT’s architecture aligns particularly well with our WebSocket optimization goals for several reasons. The model’s ability to directly process byte sequences without tokenization overhead makes it ideal for analyzing the binary streams transmitted through WebSockets. Its patch-based approach enables efficient processing of the variable-length data characteristic of financial trading applications, while the latent space representation provides a compact encoding that can potentially reduce bandwidth requirements.

The model’s demonstrated robustness to different data formats and languages also addresses a key requirement in our system, which must handle diverse data types ranging from structured financial data to user interface updates. By incorporating BLT’s architectural principles into our WebSocket implementation, we can develop a predictive encoding system that efficiently identifies redundant patterns across multiple data formats without requiring specialized encoding for each type.

3.3 Pattern Recognition in Structured Data

Recent advances in pattern recognition for structured data offer significant insights for optimizing WebSocket communications in financial trading systems. Across several key methodologies, we observe complementary approaches to modeling complex sequential and relational data patterns.

Pattern recognition in structured data, particularly for graph-based representations, provides a theoretical foundation for modeling complex data relationships. Such approaches enable superior discrimination between topologically similar structures - a capability directly applicable to modeling recurring communication patterns in WebSocket data streams. Graph-based models capture hierarchical patterns: "early layers focus on local neighborhoods while deeper layers integrate information over larger subgraphs" [15], allowing for multi-scale understanding of data dependencies.

Temporal modeling approaches introduce innovative gating mechanisms specifically designed for sequential data processing. These time-aware gates give more weight to recent activities and less to distant ones, while location-aware components focus on spatial patterns in behavior. This approach provides a framework for identifying temporal dependencies within WebSocket communications, potentially enabling more efficient prediction of data patterns based on their temporal proximity and contextual relevance.

Semi-supervised attribute-driven graph representation transforms categorical attributes into interconnected nodes within a graph structure, enabling effective pattern propagation through transaction networks. This approach demonstrates remarkable efficiency with limited labeled data, suggesting potential for rapid adaptation to evolving communication patterns in WebSocket streams with minimal training examples.

The FrameNet annotation approach offers additional techniques for template-based pattern recognition, using language models to systematically expand pattern coverage from limited examples. As Cui and Swayamdipta demonstrate, this method "replaces specific words with related words to create new templates" and "uses language models to generate new sentences" while maintaining semantic structure [6].

Lightweight dictionary learning for classification shows that "data compression techniques, specifically dictionary-based methods... can effectively capture discriminative textual patterns without relying on deep neural networks" [10]. This provides a computationally efficient approach that achieves "near state-of-the-art accuracy" while completing rapidly "using only CPU resources."

Document-level script event prediction, as explored in DocScript, analyzes "document-level narrative structures" to predict "sequential events in scripts" [5], offering insights into modeling longer sequential dependencies that may occur in WebSocket communication flows.

These methodologies collectively provide a robust theoretical framework for conceptualizing WebSocket communications as structured patterns with temporal and spatial dimensions. By representing the flow of data between server and client as a directed graph, where nodes represent data elements and edges represent transmission pathways, we can apply these pattern recognition techniques to identify redundancies and predict subsequent data elements in financial trading systems, potentially reducing bandwidth requirements while maintaining the critical real-time responsiveness needed in high-frequency trading applications.

3.3.1 Graph Isomorphism Networks: Understanding Structural Expressivity

Graph Neural Networks (GNNs) have emerged as powerful tools for modeling graph-structured data, enabling applications ranging from molecular property prediction to social network analysis. Among these models, the Graph Isomorphism Network (GIN) introduced by Xu et al. represents a significant advancement in understanding the theoretical foundations and limitations of graph neural networks. This paper addresses a fundamental question: what makes different GNN architectures more or less powerful in their ability to discriminate between distinct graph structures?

At its core, GIN establishes a compelling connection between GNNs and the Weisfeiler-Lehman (WL) graph isomorphism test, a classical algorithm used to determine if two graphs are structurally identical. This connection provides a theoretical framework for analyzing the expressive power of various GNN architectures. The authors demonstrate that standard GNN variants like GCN and

GraphSAGE, which rely on mean or max aggregation functions, cannot achieve the maximum discriminative power possible for graph representation learning.

The primary innovation of GIN lies in its architectural design, particularly in its aggregation mechanism. Unlike previous approaches, GIN employs a sum aggregator function which preserves the multiset structure of node neighborhoods, ensuring that no information is lost during feature aggregation. This is complemented by the use of multi-layer perceptrons (MLPs) that apply complex, non-linear transformations to the aggregated features. These design choices allow GIN to capture rich representations that effectively encode the structural information embedded in the WL subtree patterns.

Empirically, the authors evaluate GIN against other GNN variants across multiple graph classification datasets. Their experiments reveal that GIN consistently outperforms mean and max-pooling architectures, providing strong evidence for the theoretical claims regarding its discriminative power. Particularly noteworthy is GIN’s performance on datasets where graph structures differ subtly, demonstrating its superior ability to distinguish between topologically similar graphs.

Fundamentals of GNN Architectures

The basic operation of a GNN can be understood as follows. At each layer, a node’s feature representation is updated by combining its current state with a summary of its neighbors’ features. Figure 3.9 illustrates this process. Here, the network learns to represent multi-hop relationships in the graph through repeated aggregations.

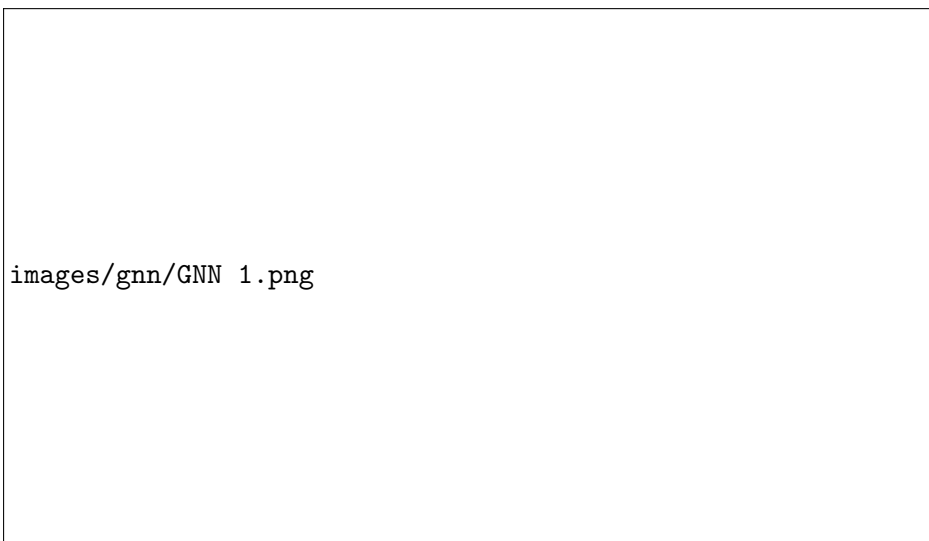


Figure 3.9: Schematic overview of the neighborhood aggregation process in a GNN. Each node gathers and aggregates features from its neighbors to form a refined representation.

Xu et al. showed that the expressivity of GNNs is closely related to the Weisfeiler-Lehman (WL) graph isomorphism test. In particular, the Graph Isomorphism Network (GIN) utilizes a *sum aggregator*—a function that, when paired with multi-layer perceptrons (MLPs), can achieve the theoretical maximum discriminative power (i.e., as powerful as the WL test) [?]. This property makes GIN especially effective in distinguishing graphs with subtle structural differences.

Key Innovations and Empirical Performance

The discriminative power of GNNs largely depends on the choice of aggregation function. As shown in Figure 3.10, sum aggregation (as used in GIN) preserves the full multiset of neighbor features, unlike mean or max pooling which can lose crucial multiplicity information. This difference directly translates into enhanced performance in tasks where fine-grained structural distinctions are essential.

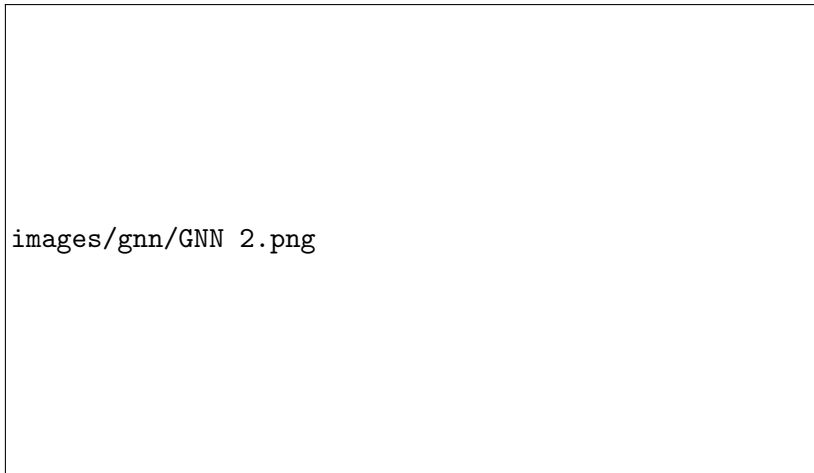


Figure 3.10: Comparison of aggregation functions in GNNs. Sum aggregators retain full neighborhood information, yielding superior discriminative power over mean and max pooling.

Moreover, deeper GNN layers can capture increasingly global properties of the graph. Figure 3.11 provides a visual representation of how early layers capture local features while later layers integrate global structural information. This multi-scale representation is particularly useful when modeling complex communication patterns.

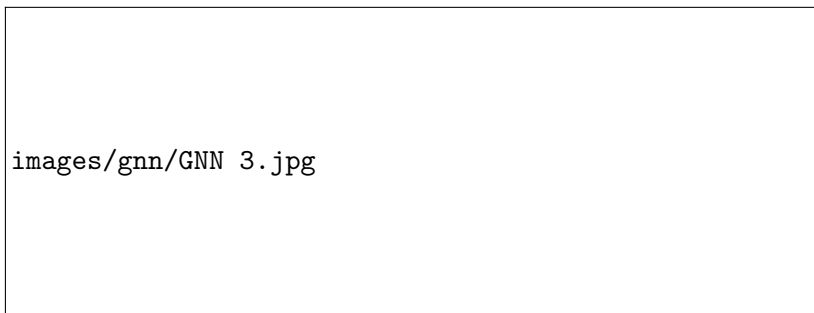


Figure 3.11: Visualization of hierarchical feature extraction in GNNs. Early layers focus on local neighborhoods; deeper layers integrate information over larger subgraphs.

3.3.2 Application in WebSocket Communication Optimization

In our application, we model the flow of data between the server and client as a directed graph, where nodes represent data elements and edges represent transmission pathways. GNNs enable us to detect redundancies, predict missing data and improve latency.

Figure 3.12 illustrates the concept of applying GNNs to model and optimize data transmission in our WebSocket architecture.



Figure 3.12: Application of GNNs in optimizing WebSocket communications. The model identifies and compresses predictable data patterns, reducing bandwidth and improving latency.

For our work on optimizing WebSocket communications, GIN's approach to understanding graph structures offers valuable insights. By conceptualizing the flow of data between server and client as a directed graph, we can potentially apply GIN's discriminative capabilities to identify recurring patterns in communication sequences. This pattern recognition could enable more efficient prediction and compression of data paths, potentially reducing bandwidth requirements while maintaining the integrity of the information being transmitted.

3.3.3 Spatial-Temporal Gated Network for Fraud Detection

The Spatial-Temporal Gated Network (STGN) presented by Zheng et al. addresses a critical challenge in the financial sector: detecting fraudulent credit card transactions within an imbalanced classification context. This paper recognizes that existing approaches often fail to effectively capture both the temporal dynamics and spatial patterns inherent in transaction data, leading to sub-optimal fraud detection performance.

Traditional fraud detection methods typically use either raw transaction features or simple aggregated features, which may not fully capture the nuanced characteristics of fraudulent behavior. Moreover, many sequential models incorrectly assume equal influence from all previous transactions regardless of time gaps between them. STGN addresses these limitations through a specialized recurrent architecture designed to learn informative representations of transactions by accounting for both spatial and temporal dimensions simultaneously.

At the heart of STGN is a sophisticated recurrent unit featuring two innovative components: a time-aware gate and a location-aware gate. The time-aware gate allows the model to consider temporal intervals between transactions, giving more weight to recent activities and less to distant ones. This mimics how fraud analysts typically place greater emphasis on recent behavior changes. Complementing this, the location-aware gate helps the model focus on spatial patterns in transaction behavior, particularly useful for detecting geographically anomalous activities often associated with fraud.

The authors enhance this architecture with an attention mechanism that selectively focuses on the most relevant past transactions when analyzing a current one. This attention-based approach enables the model to identify which historical transactions provide the most valuable context for assessing the current transaction. Additionally, the model employs a representation interaction component that combines user history, current transaction details, and contextual information to generate a comprehensive transaction representation for the final fraud prediction layer.



Figure 3.13: Gated mechanism images from Yu Xie's work, illustrating how the network fuses spatial and temporal features to enhance fraud detection.

The STGN model demonstrates superior performance compared to traditional approaches, achieving significantly higher accuracy and fraud detection rates while maintaining reasonable computational requirements. Particularly impressive is its ability to handle the severe class imbalance typical in fraud detection scenarios, where legitimate transactions vastly outnumber fraudu-

lent ones.

For our WebSocket optimization work, STGN's approach to handling sequential data with temporal and spatial components offers valuable insights. The concept of weighting the importance of previous elements in a sequence based on their recency and relevance could be adapted to prioritize certain data patterns in WebSocket communications. By identifying which communication patterns are most predictive of subsequent data flows, we could potentially develop a more efficient compression and prediction mechanism that reduces bandwidth usage while maintaining application responsiveness.

3.3.4 Semi-supervised Credit Card Fraud Detection via Attribute-Driven Graph Representation

In the domain of financial transaction analysis, the semi-supervised credit card fraud detection framework presents a significant advancement in addressing three fundamental challenges that plague conventional approaches. First, traditional methods primarily rely on labeled transaction data, effectively ignoring the wealth of fraud patterns that exist within unlabeled transactions. Second, existing approaches demonstrate limited capacity to leverage categorical attributes—such as card type and merchant location—which are critical for robust fraud pattern recognition. Finally, current graph-based methods encounter significant scalability issues when confronted with real-world transaction volumes and often fail to effectively propagate risk information through transaction networks.

The methodology introduced in this research employs a semi-supervised learning paradigm that capitalizes on both labeled and unlabeled transaction data. This approach is particularly valuable in the fraud detection domain, where obtaining labeled examples is both expensive and time-consuming. By leveraging the significantly larger pool of unlabeled transactions, the model can extract valuable patterns that would otherwise remain undetected by supervised learning approaches alone.

At the core of this framework lies the attribute-driven graph representation, which fundamentally transforms how categorical attributes are utilized in fraud detection models. Rather than processing each attribute as an independent feature—the approach taken by most conventional methods—this technique models them as interconnected nodes within a graph structure. This representation enables the model to capture complex fraud patterns that manifest through relationships between different attribute combinations, substantially enhancing detection capabilities.

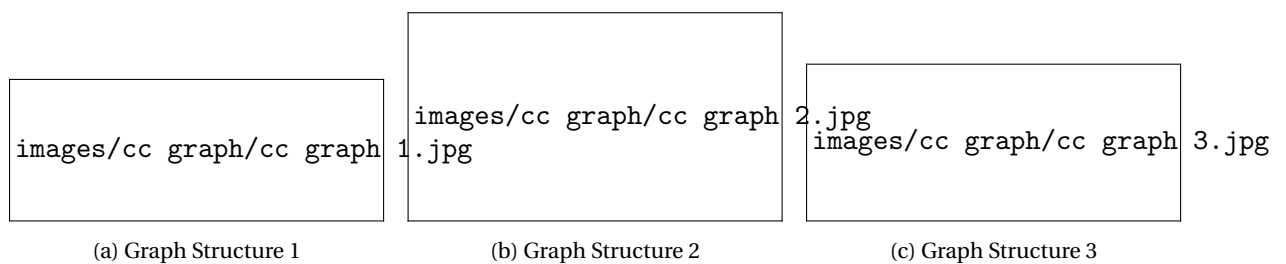


Figure 3.14: Temporal transaction graph examples from Sheng Xiang et al. Each subfigure shows a different aspect of the graph construction that captures transaction sequences.

The graph construction process is particularly noteworthy for its attention to temporal dynamics. Transactions are represented as nodes in a temporal sequence, with edges connecting consecutive transactions from the same card within defined time windows (typically 24 hours). This structure allows the model to capture the evolution of transaction patterns over time, a critical factor in identifying anomalous behavior that might indicate fraudulent activity.

A key innovation in this approach is the risk propagation mechanism that enables information flow through the transaction network. By conceptualizing confirmed fraudulent transactions

as origin points (analogous to "patient zero" in epidemiological models), the system traces connections through the network and adjusts risk scores based on transaction proximity and similarity. This propagation is implemented through a specialized message-passing neural network that preserves attribute relationships while disseminating risk signals.

The framework incorporates an anti-leakage masking technique during training, where a percentage of labels (typically 10%) are randomly concealed. This forces the model to develop robust pattern recognition capabilities rather than simply memorizing labeled examples—a technique that significantly improves generalization to new, unseen transactions.

Empirically, this approach demonstrates remarkable performance, achieving up to 76.16% fraud detection accuracy with substantially reduced false positive rates compared to traditional supervised systems. Particularly impressive is the model's ability to maintain strong performance even when operating with as little as 10% labeled data, making it exceptionally well-suited for real-world deployment scenarios where comprehensive labeling is impractical.

For our research on optimizing WebSocket communications, this semi-supervised approach offers valuable insights into pattern detection and propagation in structured data flows. The concept of attribute-driven graph representation could be adapted to model the transmission patterns in financial data streams, potentially enabling more efficient prediction and compression mechanisms. By identifying which communication patterns are most predictive of subsequent data flows, we could develop compression strategies that reduce bandwidth requirements while maintaining the responsiveness critical for financial trading applications.

3.3.5 Annotating FrameNet via Structure-Conditioned Language Generation

Expanding FrameNet annotations has long been a challenge in semantic role labeling (SRL), primarily due to the resource-intensive nature of manual annotation and the limited coverage of existing lexical units (LUs). FrameNet, a linguistic database grounded in frame semantics, provides a structured representation of situations, events, and their participants, but only 62% of LUs are annotated, leaving significant gaps in its utility for downstream tasks such as machine translation, information extraction, and event recognition. Existing automatic methods, such as sister LU replacement, often result in semantic inconsistencies that limit scalability while compromising semantic accuracy.

To address these challenges, Cui and Swayamdipta propose a novel framework leveraging large language models (LLMs), such as T5 and GPT-4, to generate frame-semantic annotations through structure-conditioned language generation. Their approach introduces an overgenerate-and-filter methodology that ensures semantic consistency by conditioning generation on frame structures and filtering out inconsistent outputs. This method not only improves annotation quality but also supports low-resource settings by efficiently expanding FrameNet coverage.

The framework begins with existing annotated sentences from FrameNet. Specific lexical units (LUs) are replaced with related words using predefined templates. The system identifies parts of the sentence that might need modification due to word replacement and employs LLMs to generate multiple new sentences based on these templates. A filtering mechanism then discards outputs that fail to maintain semantic fidelity or coherence. Figure 3.15 illustrates the step-by-step process, including LU replacement, structure-conditioned generation, and filtering for consistency.

Empirical evaluations demonstrate the effectiveness of this approach across multiple metrics. Table 3.4 summarizes key results comparing unaugmented data with augmented data generated using T5 and GPT-4 conditioned on frame structures and frame elements (FE). Augmentation with GPT-4 | Frame+FE achieved the best performance with an F1 score improvement over unaugmented settings.

Filtering mechanisms further improved semantic fidelity and human acceptability of generated sentences while maintaining diversity in outputs measured via Self-BLEU scores (lower Self-BLEU indicates higher diversity). Table 3.5 highlights improvements in perplexity, FE fidelity, and human acceptance rates before and after filtering.

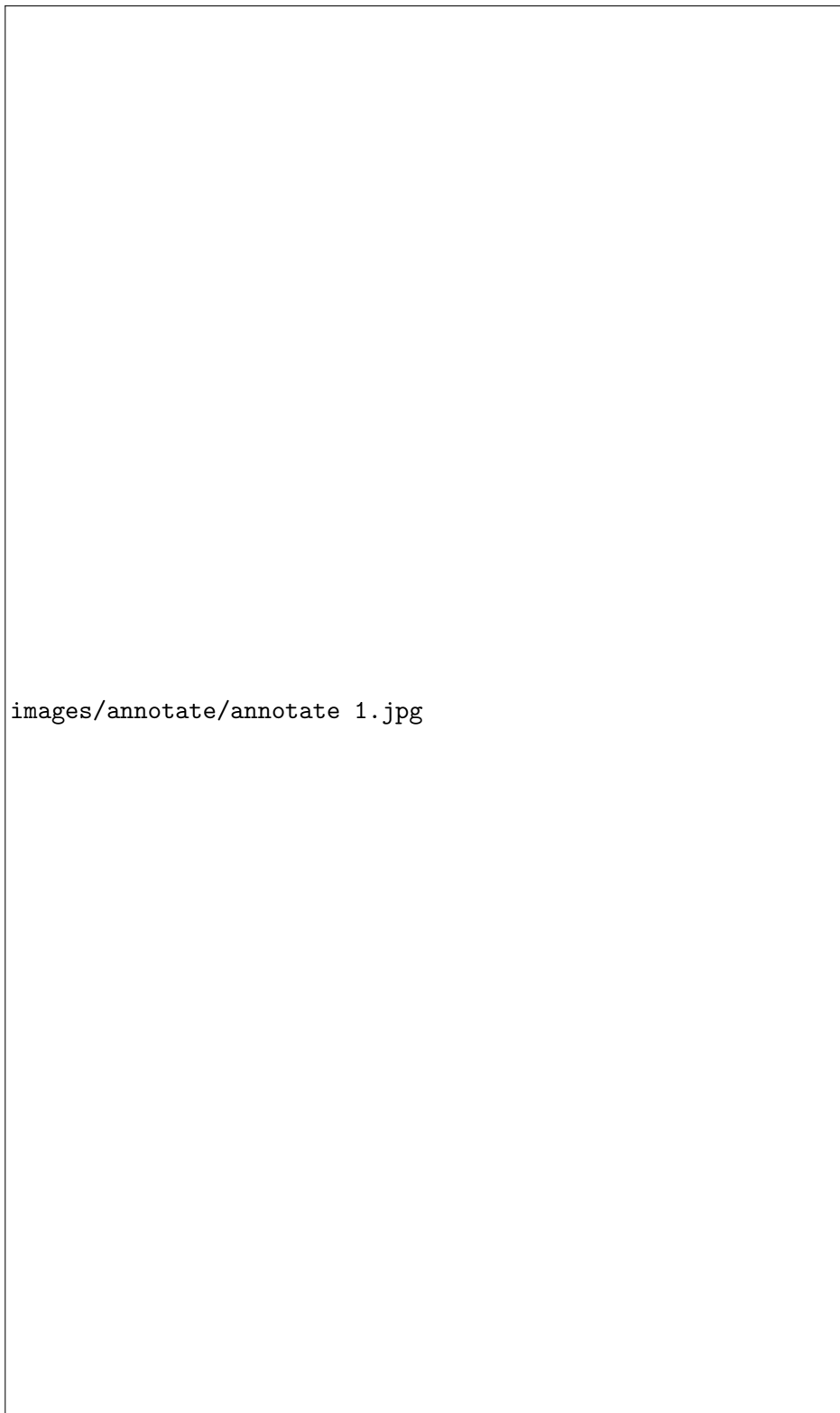


Figure 3.15: Framework for annotating FrameNet via structure-conditioned language generation. The process involves LU replacement, structure-conditioned generation using T5/GPT-4, and filtering inconsistent outputs.

The study also demonstrates diminishing returns when adding more human-annotated training data for SRL tasks beyond a certain threshold, emphasizing the importance of high-quality or diverse data rather than sheer quantity.

For our WebSocket optimization project at smartTrade, this structured language generation

Setting	All LUs F1	Augmented LUs F1
Unaugmented	0.677 $\pm$ 0.004	0.681 $\pm$ 0.012
Augmented w/ T5 FE	0.683 $\pm$ 0.000	0.682 $\pm$ 0.006
Augmented w/ GPT-4 Frame+FE	0.684 $\pm$ 0.002	0.677 $\pm$ 0.010

Table 3.4: F1 scores of all LUs and augmented LUs under various settings averaged across three random seeds. Best results are bolded.

Model	FE Fidelity	Perplexity	Human Acceptability
Before Filtering (T5 FE)	0.862	127.6	0.711
After Filtering (GPT-4 Frame+FE)	0.733	111.8	0.821

Table 3.5: Impact of filtering on FE fidelity, perplexity, and human acceptability for T5 and GPT-4 generations conditioned on frame structures and frame elements (FE).

approach offers valuable insights into pattern recognition within structured data flows. The ability to condition generative AI models on predefined structures aligns closely with our goal of dynamically identifying recurring patterns in financial transaction data streams for predictive compression. By adopting similar overgenerate-and-filter methodologies tailored to WebSocket communication patterns, we can enhance bandwidth efficiency while maintaining real-time responsiveness critical to client-server applications.

3.4 Code Understanding and Performance Optimization

Recent advances in code understanding models and performance optimization techniques offer complementary approaches to our WebSocket optimization challenge. In the domain of code understanding and generation, CodeT5 [7] represents a significant advancement with its unified encoder-decoder framework that specifically addresses the unique characteristics of source code. Unlike general language models, CodeT5 incorporates identifier-aware pre-training objectives that capture the semantic relationships between function names, variable definitions, and their usage patterns within code. This specialized approach yields substantial improvements over previous models—achieving state-of-the-art performance on the CodeXGLUE benchmark [9] with gains of 8.0-10.5% on code summarization and 17.9% on code generation tasks. A particularly relevant aspect of CodeT5’s architecture is its ability to effectively process structured, syntax-dependent data while maintaining context across long sequences, a capability directly applicable to optimizing the structured data transmissions in our WebSocket implementation.

The performance optimization domain offers complementary techniques focused on reducing computational overhead and bandwidth requirements. The JavaScript classification approach presented by Chaqfeh et al. [11] demonstrates how machine learning classifiers can identify potentially performance-degrading JavaScript components with 91% accuracy, enabling on-the-fly optimization decisions that reduce mobile page load times by up to 42%. This approach is particularly relevant to our client-side WebSocket implementation, where rapid classification of incoming data patterns could enable more efficient rendering of financial trading information. Complementing this classification approach, the lightweight conceptual dictionary learning technique introduced by Wan et al. [10] achieves "near state-of-the-art classification accuracy while maintaining high computational efficiency" by leveraging the LZW compression algorithm to identify and encode recurring patterns within text data. Their method creates conceptual dictionaries that capture domain-specific terminology and structural patterns without requiring deep neural networks, making it particularly suitable for resource-constrained environments.

When integrated with code-aware models like CodeT5, these lightweight classification techniques offer a promising pathway for WebSocket optimization. The CodeT5 model can provide the deep understanding of structured data patterns necessary for predictive encoding, while the LZW-based dictionary approach offers an efficient mechanism for identifying and compressing

recurring patterns at runtime. Sîrbu’s work on deobfuscating JavaScript through character-based tokenization [12] further demonstrates how byte-level processing can effectively handle code-like structures even when deliberately obfuscated, achieving "94.6% accuracy in identifying malicious patterns" with minimal computational overhead. This suggests that even when financial data streams contain complex or unusual patterns, our approach can maintain high prediction accuracy.

These combined approaches address the central challenge in our WebSocket optimization problem: identifying predictable patterns in structured data transmissions without introducing prohibitive computational overhead. By leveraging CodeT5’s understanding of code structures alongside efficient classification techniques, we can develop a system that selectively transmits only unpredictable components while maintaining the real-time responsiveness critical for financial trading applications. The primary limitation of existing approaches is their lack of optimization for the specific temporal patterns and data structures found in financial trading applications—a gap our research aims to address through specialized fine-tuning and hybrid architecture design.

3.4.1 CodeT5

In the domain of code understanding and generation, Wang et al. introduced CodeT5, a unified encoder-decoder model that addresses fundamental limitations in existing approaches. Unlike previous models that employed either encoder-only architectures (e.g., BERT, RoBERTa) optimized for understanding tasks or decoder-only architectures (e.g., GPT) designed for generation tasks, CodeT5 leverages a unified framework capable of excelling at both types of tasks simultaneously. This versatility is particularly valuable for applications requiring both code analysis and synthesis capabilities.

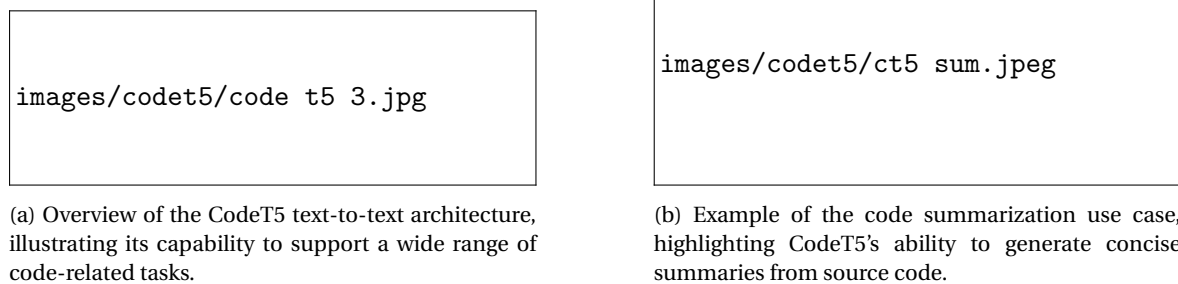


Figure 3.16: CodeT5 architecture alongside a representative use case (code summarization).

The model’s architecture is rooted in the T5 framework but introduces two key innovations tailored specifically for programming language processing. First, CodeT5 implements an identifier-aware pre-training objective that enhances the model’s understanding of code semantics by specifically focusing on program identifiers (e.g., variable names, function names). This approach recognizes that identifiers contain critical semantic information in code that traditional token-based models often overlook. Second, the model incorporates a bimodal dual generation mechanism that improves alignment between natural language and programming language through bidirectional conversion between code and comments.

CodeT5 was pre-trained on the CodeSearchNet dataset comprising approximately 8.35M instances across six programming languages, with additional C/C# data collected from GitHub repositories. The authors employed a custom Byte-Pair Encoding (BPE) tokenizer optimized for code, reducing sequence lengths by 30-45% compared to standard tokenizers. This pre-training regimen included multiple objectives: masked span prediction, identifier tagging, masked identifier prediction, and bimodal dual generation tasks.

Empirical evaluations demonstrated that CodeT5 achieves state-of-the-art performance across

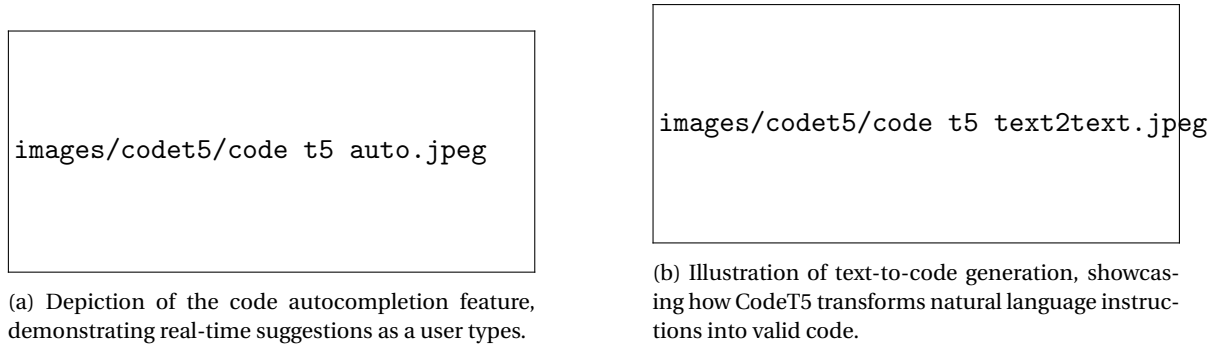


Figure 3.17: Two additional CodeT5 use cases: code autocompletion and text-to-code generation.

multiple code-related tasks including code defect detection, clone detection, code summarization, generation, translation, and refinement. The model's ability to understand identifier relationships and structure within code makes it particularly relevant to our work on optimizing data transmission patterns, as it demonstrates how specialized attention to structural elements (identifiers in code, patterns in data streams) can significantly enhance model performance in domain-specific applications.

3.4.2 Accelerating Mobile Web Performance

Presents an ML-driven approach for JavaScript classification integrated into browsers:

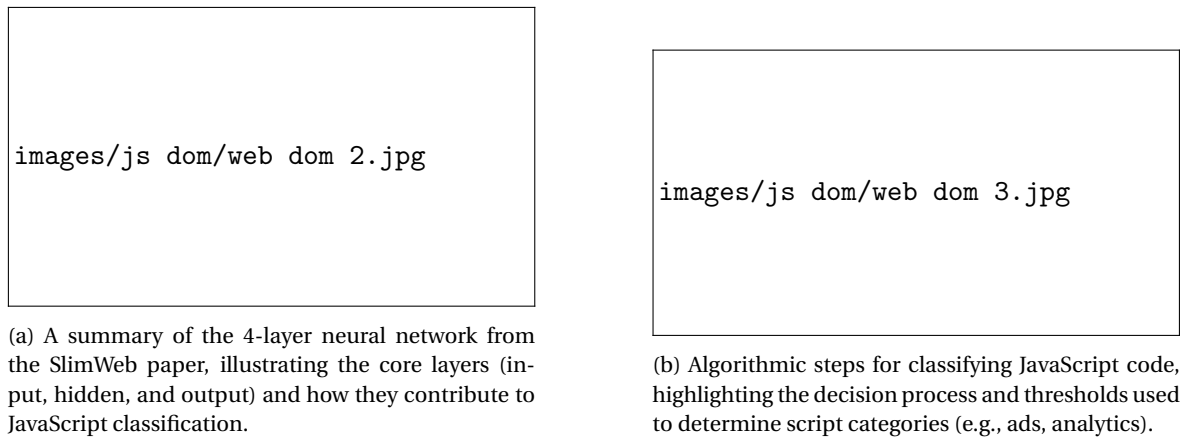


Figure 3.18: Two key components of the SlimWeb approach to JavaScript classification: (a) a 4-layer neural network architecture for extracting and learning features, and (b) an explicit algorithm detailing how scripts are categorized based on performance-critical criteria.

Optimizing web performance, particularly in real-time financial trading platforms, is crucial due to stringent latency and responsiveness requirements. JavaScript (JS) significantly contributes to web page size and computational overhead, often resulting in slower load times, especially on resource-constrained devices. Traditional methods for handling JavaScript inefficiencies—such as rule-based simplifiers or server-dependent optimizations—either lack adaptability or introduce additional latency due to constant server communication.

To address these limitations, recent research by Zhang et al. proposes an innovative machine learning-driven approach that classifies JavaScript elements directly within the browser environment. The primary objective of this method is to distinguish between critical and non-critical JS scripts at runtime, enabling selective loading and execution. Their methodology involves training a supervised neural network classifier on a dataset comprising approximately 127,000 JS elements collected from over 20,000 web pages. This classifier categorizes scripts into predefined functional categories (e.g., content-critical scripts versus advertisements or analytics scripts) with

a precision-recall accuracy of around 90%.

The implementation integrates this classification model into a modified version of the DuckDuckGo browser, allowing real-time decisions about script execution without external server interactions. Additionally, the approach employs a caching mechanism that stores classification results locally, further reducing computational overhead during repeated page visits.

Empirical evaluations demonstrate significant performance improvements: page load times decreased by up to 30%, while total JavaScript payload sizes reduced by approximately 23.7%. Crucially, these optimizations were achieved without compromising core page functionality or user experience.

The implications of this research are particularly relevant to our work on optimizing WebSocket-based communications at smartTrade. The concept of dynamically classifying and selectively transmitting data based on its criticality aligns closely with our goal of minimizing transmitted data volume while maintaining real-time responsiveness. By applying similar classification techniques to structured data models before their binary transformation for WebSocket transmission, we can potentially achieve substantial bandwidth savings and latency reductions. Furthermore, the demonstrated effectiveness of client-side caching mechanisms highlights an additional avenue for optimization within our proposed system architecture.

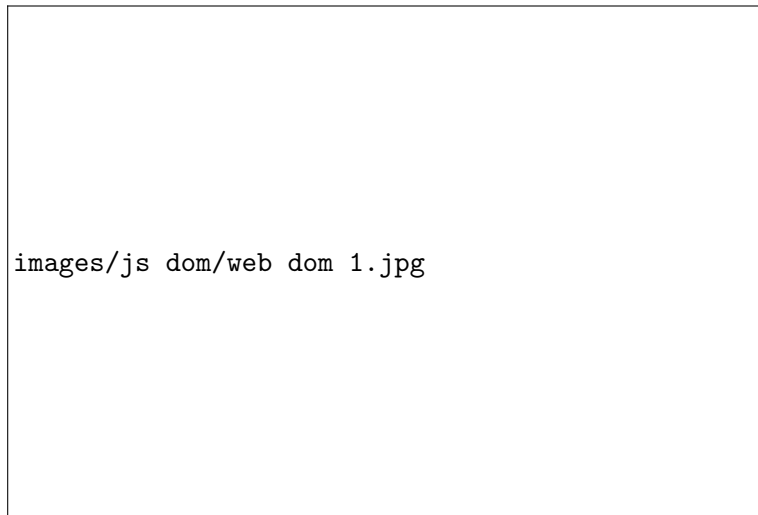


Figure 3.19: High-level browser architecture for script classification. The ML classifier determines whether a JavaScript file is critical or non-critical; non-critical scripts are blocked to reduce overhead, while essential scripts are allowed for proper page functionality.

3.4.3 Lightweight Text Classification Using LZW Compression

Efficient text classification methods are crucial for real-time applications, particularly in domains such as financial trading systems where latency and computational efficiency directly impact performance. Traditional text classification approaches often rely on complex neural architectures or resource-intensive embeddings, making them less suitable for scenarios demanding rapid inference with limited computational resources. To address these limitations, recent research by Jiang et al. introduces a novel approach leveraging the Lempel-Ziv-Welch (LZW) compression algorithm for lightweight text classification tasks.

The primary objective of their study is to explore whether data compression techniques, specifically dictionary-based methods like LZW, can effectively capture discriminative textual patterns without relying on deep neural networks or pre-trained embeddings. The authors propose a straightforward yet innovative methodology: first, they apply the LZW algorithm to scan textual data and generate an initial dictionary of frequent patterns. Subsequently, this dictionary is refined based on its discriminative power—patterns that strongly correlate with specific classes are retained,

while less informative entries are discarded. The resulting refined dictionary serves as a compact representation of essential textual features.

In their experimental setup, texts are transformed into sparse vector representations based on the presence or absence of these discriminative patterns. These vectors are then classified using simple and computationally inexpensive classifiers such as Support Vector Machines (SVM) or shallow neural networks. Empirical evaluations demonstrate that this approach achieves near state-of-the-art accuracy on simple datasets, with only a minor performance gap (approximately 2%) compared to more complex models. Remarkably, the method exhibits exceptional performance—up to 98% accuracy—on datasets characterized by limited vocabulary size or repetitive linguistic structures. However, it faces challenges when applied to highly complex datasets with extensive vocabularies and nuanced semantics, resulting in accuracy reductions of approximately 40-50%.

A significant advantage of the proposed method is its computational efficiency: the entire training process completes rapidly (approximately 67 seconds) using only CPU resources without requiring GPUs or pre-trained word embeddings. This efficiency makes it particularly attractive for deployment in latency-sensitive environments such as financial trading platforms.

The lightweight compression-based classification approach provides valuable insights: The concept of using dictionary-based compression techniques aligns closely with our goal of efficiently encoding structured data models before binary transmission over WebSockets. By identifying recurring patterns within structured communication sequences—analogueous to discriminative textual patterns—we can potentially develop compact representations that significantly reduce bandwidth usage and improve client-side latency.

Furthermore, this study highlights the potential benefits of combining traditional data compression algorithms with simple machine learning classifiers as an alternative to more resource-intensive neural architectures. Such hybrid approaches could be particularly beneficial in our context, offering a balance between computational efficiency and predictive accuracy suitable for real-time financial trading applications.

Chapter 4

Implementation & Optimization Plan

To illustrate the practical applications of our research, we systematically applied our concepts and analyzed the outcomes to determine their suitability for our use case. Initially, by employing the Byte Latent Transformer, we demonstrated that predicting the subsequent byte in a text sequence can establish meaningful connections.

Furthermore, we executed a case study utilizing WebSocket communications between a Java server and web browsers. In this investigation, we incorporated the Pony SDK developed by Smart Trade and implemented the application known as Sample Spring.

4.1 Implementation Context

In an environment where data flows between a Java server and web browsers using the WebSocket protocol, efficient bandwidth management is essential. Rather than compressing binary data directly, our implementation focuses on compression at the level of structured data models (primitive types, strings, arrays, etc.) before transforming them into a binary stream.

4.2 System Architecture

The system architecture consists of two main components: the **Server** and the **Client**, which communicate via WebSocket. The server handles the data processing and communication, while the client interacts with the user interface and integrates JavaScript frameworks. The architecture can be described as follows:

4.2.1 Server

The server performs the following functions:

1. Request handling: continuously listens for and processes incoming client requests.
2. WebSocket communication: establishes and maintains a persistent, bidirectional communication channel with the client.
3. Data processing: executes encoding and decoding operations to ensure data integrity and minimal latency.

4.2.2 Client

The client is designed to:

1. Render content: dynamically displays data and visual content to the user.
2. Enable real-time interaction: maintains a WebSocket connection for continuous data exchange.

3. Support framework integration: leverages JavaScript frameworks via PAddons for modular and scalable front-end development.

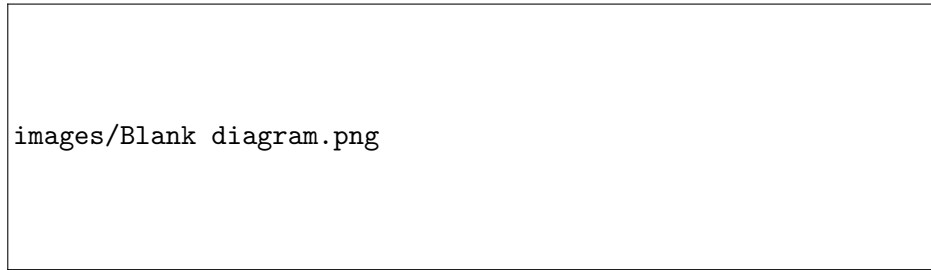


Figure 4.1: System Architecture Diagram: Server and Client Communication via WebSocket

4.3 Process Flow

The system operates through the following steps:

1. Application generates model-value pairs to send to client
2. WebSocket.encode intercepts the data before transmission
3. TransformerPredictor analyzes patterns and predicts/compresses data
4. Only unpredictable data is encoded for transmission
5. WebSocketPusher sends the optimized data to client

4.4 PonySDK Implementation Details

4.4.1 Static Dictionary

Utilizing a static dictionary approach, as seen in the PonySDK implementation (specifically at `Websocket.java`, line 333), we test scenarios where known data values present in the dictionary are represented by concise codes, and unknown values are sent in full.

4.4.2 Dynamic Dictionary Model

We further explore a more sophisticated dynamic dictionary model, aiming to create a system capable of detecting known paths in a directed graph structure, associating these node sets with specific keys. This approach necessitates careful determination of key construction, likely involving sets of key/value pairs.

Error handling will involve scenarios where incorrect paths are detected, in which the system can either:

1. Notify of the mistake and resend the entire instruction.
2. Send only the generated nodes and any nodes that differ from the expected paths.

We hypothesize that transformer-based generative models may effectively predict the subsequent nodes in these graphs, thus improving prediction accuracy and system efficiency.

The PonySDK source code used for this project is publicly available and can be accessed here: [PonySDK GitHub Repository](#).

4.5 Code Example

```
// In WebSocket.java for sending an
// encoded message:
public void
encode(ServerToClientModel
model, Object value)

@Override
public void
onWebSocketMessage(String
message) {
    // Decode the incoming JSON
    // message and update the UI
}

// Initialization of the server and
// application context
PonySDKServer server = new
PonySDKServer(new
ApplicationConfiguration(/*...*/));
server.setPort(8080);
```

4.6 Optimization Plan

1. Compress structured data models (e.g., primitive types, strings, arrays) before transforming them into binary streams.
2. Use transformer-based generative models to identify repetitive sequences dynamically.
3. Notify of incorrect paths and resend instructions or generate differing nodes only.

4.7 PonySDK Code Snippets

```
> Task :ponysdk:compileJava UP-TO-DATE
> Task :ponysdk:processResources UP-TO-DATE
> Task :ponysdk:classes UP-TO-DATE
> Task :ponysdk:gtwc UP-TO-DATE
> Task :ponysdk:jar
> Task :sample:compileJava UP-TO-DATE
> Task :sample:processResources UP-TO-DATE
> Task :sample:classes UP-TO-DATE
> Task :sample:runSampleSpring
```

```
OpenJDK 64-Bit Server VM warning: Option UseConcMarkSweepGC was deprecated
  in
version 9.0 and will likely be removed in a future release.
Listening for transport dt_socket at address: 8888
06:47:57.659 INFO  main [log] Logging initialized @1030ms to
org.eclipse.jetty.util.log.Slf4jLog
06:47:57.726 INFO  main [PonySDKServer] Adding application #sample
06:47:57.752 INFO  main [Server] jetty-9.4.43.v20210629; built:
  2021-06-30T11:07:22.254Z;
```

```

jvm 11.0.26+0
06:47:57.772 INFO  main [session] DefaultSessionIdManager workerName=node0
06:47:57.774 INFO  main [session] node0 Scavenging every 600000ms
06:47:57.793 INFO  main [ContextHandler] Started
  - ServletContextHandler{/sample,null,AVAILABLE}
06:47:57.815 INFO  main [AbstractConnector] Started
  - ServerConnector{HTTP/1.1}{0.0.0.0:80}
06:47:57.838 INFO  main [SslContextFactory]
  - x509=X509@63429932(jetty,h=[ciaravola nicolas])
for
  - Server@6f012914[keyStore=file:///Users/shubhankar/Documents/GitSpace/PonySDK/keystore]
06:47:57.921 INFO  main [AbstractConnector] Started
  - ServerConnector{SSL}{0.0.0.0:8082}
06:47:57.921 INFO  main [Server] Started @1295ms
06:47:57.921 INFO  main [PonySDKServer] Webserver started on: 127.0.0.1:80

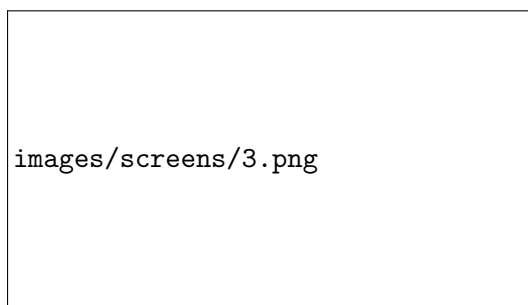
```



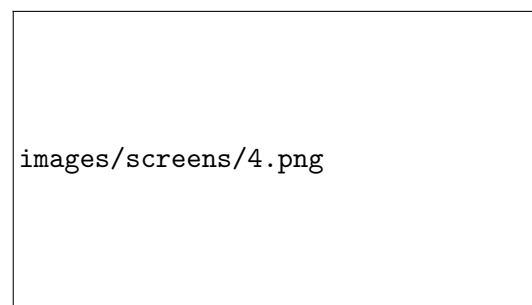
(a) Initial UI state of Sample Spring with "Send Custom UI Components/Create Dynamic Components".



(b) Initial UI state of Sample Spring.



(c) Same updated UI, with console logs displayed.



(d) UI after clicking "Send UI Components".

Figure 4.2: Screenshots from the Sample Spring application. (a) The initial minimal UI with the button to send UI Components. (b) The second half of the page before information is sent. (c) The same updated UI with the browser console open, displaying logs. (d) final UI state on the same page.

4.7.1 Analysis of Current WebSocket Implementation

After examining the `WebSocket.java` file in the PonySDK framework, several critical implementation aspects emerge that directly impact client-side latency and provide opportunities for optimization using generative AI approaches.

WebSocketPusher Configuration

The WebSocketPusher is initialized with specific size constraints that establish the bandwidth limitations of the system:

```
this.websocketPusher = new WebSocketPusher(session, 1 << 20, 1 << 12,
                                           TimeUnit.SECONDS.toMillis(60));
```

This configuration sets a 1MB (1 « 20) total buffer size and 1KB (1 « 12) maximum chunk size, revealing explicit bandwidth optimization concerns. As noted in the code comments: “Don’t set max chunk size > 8K because when using Jetty WebSocket compression, the chunks are limited to 8K.” These constraints directly impact our ability to transmit large data streams efficiently.

Sequential Encoding Pattern

The current implementation encodes each model-value pair sequentially, creating significant overhead when transmitting repetitive data structures:

```
@Override
public void encode(final ServerToClientModel model, final Object value) {
    // ...
    websocketPusher.encode(model, value);
    // ...
}
```

This pattern is particularly evident in component transmission methods:

```
public void sendUIComponent(String componentType, String componentId, String componentText) {
    beginObject();
    encode(ServerToClientModel.UI_COMPONENT_TYPE, componentType);
    encode(ServerToClientModel.UI_COMPONENT_ID, componentId);
    encode(ServerToClientModel.UI_COMPONENT_TEXT, componentText);
    endObject();
    flush();
}
```

Each element is encoded individually without considering recurrent patterns or contextual prediction opportunities that our generative AI approach could exploit.

Latency Tracking Mechanisms

The code explicitly tracks various types of latency, highlighting its critical importance in the system:

```
final long roundtripLatency = TimeUnit.MILLISECONDS.convert(
    System.nanoTime() - lastSentPing, TimeUnit.NANOSECONDS);
uiContext.addRoundtripLatencyValue(roundtripLatency);
uiContext.addTerminalLatencyValue(terminalLatency);
uiContext.addNetworkLatencyValue(networkLatency);
```

This built-in latency measurement framework provides an ideal foundation for evaluating our generative AI optimization approach, offering direct comparative metrics for before-and-after performance assessment.

Real-time Requirements

The implementation's heartbeat mechanism demonstrates strict real-time requirements:

```
private void sendHeartbeat() {
    beginObject();
    encode(ServerToClientModel.HEARTBEAT, null);
    endObject();
    flush0();
}
```

Any optimization solution must maintain these real-time guarantees while reducing bandwidth usage. Our generative AI approach must be lightweight enough on the client side to avoid introducing delays that might disrupt this timing-sensitive communication.

Error Handling Criticality

The error handling approach emphasizes the mission-critical nature of the WebSocket connection:

```
try {
    websocketPusher.flush();
} catch (final IOException e) {
    log.error("Can't write on the websocket for #{}, so we destroy the application",
        uiContext.getID(), e);
    uiContext.onDestroy();
}
```

This demonstrates that communication failures are considered fatal to the application, underscoring the importance of reliable data transmission in our optimization approach.

These core elements of the WebSocket implementation represent both challenges and opportunities for our generative AI-based optimization. By targeting the sequential encoding pattern while respecting the real-time requirements and error handling needs, our approach can reduce bandwidth usage without compromising application reliability.

4.8 Byte Latent Transformer Experiment

To evaluate the potential of byte-level processing for our technical assistance system, we conducted an experiment using the Byte Latent Transformer (BLT) architecture. This experiment aimed to assess BLT's performance on real-world data from smartTrade, focusing on its ability to process and generate text at the byte level.

Dataset and Preprocessing

We utilized a dataset of articles, each containing a label, title, and description. The preprocessing steps included:

1. Removal of non-ASCII characters
2. Normalization of whitespace
3. Conversion of CSV data to individual text files
4. Organization of files using an "{index}_{label}.txt" naming convention

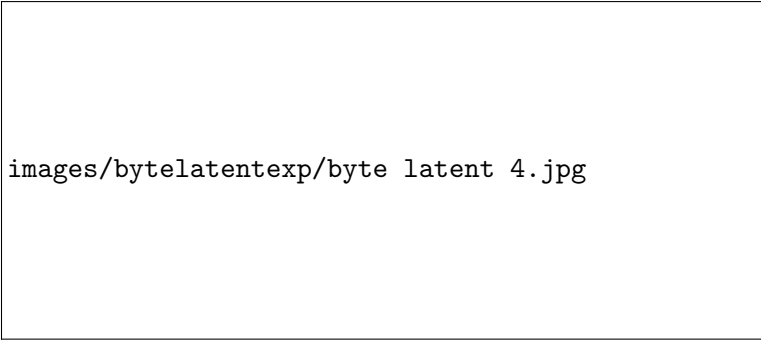


Figure 4.3: A sample CSV table containing article entries with labels, titles, and descriptions.



Figure 4.4: A historical newspaper snippet (e.g., from The New York Times) showing a map related to the 18th Amendment adoption.

Model Architecture and Training

The BLT model was implemented with the following key components:

- 1. Byte-level input processing
- 2. Patch-based encoding
- 3. Latent Global Transformer
- 4. Local Decoder for byte reconstruction

Training was conducted using a ByteDataset and DataLoader, with the model learning to predict subsequent bytes in the sequence.

Performance Analysis

After training, we observed the following key metrics:

- 1. Training Efficiency: Average time per batch was approximately 1.44 seconds.
- 2. Loss Convergence: Training loss decreased from 3.44 to 2.71, while evaluation loss improved from 2.95 to 2.67.
- 3. Perplexity: Showed significant improvement, with training perplexity reducing from 44.25 to 15.13, and evaluation perplexity from 19.24 to 14.54.

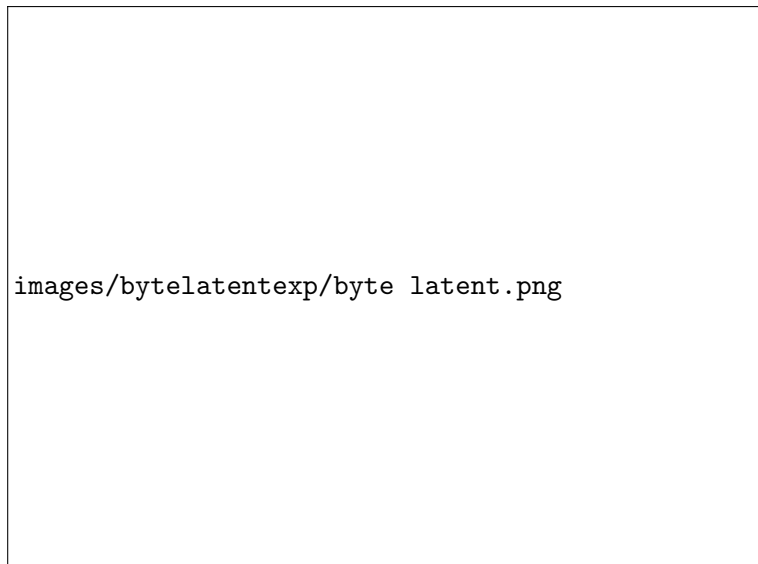


Figure 4.5: The Byte Latent Transformer architecture, including a Local Encoder, a Latent Transformer, and a Local Decoder for byte-level processing.

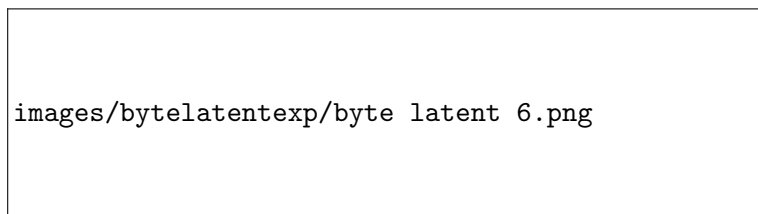


Figure 4.6: Command-line output of the initial training phase (e.g., using `python train-gen.py`), indicating device usage and initial loss.



Figure 4.7: Subsequent training logs showing updates to train and evaluation loss, perplexity, and accuracy, as well as an error message regarding `eval_loss`.

4. Accuracy: Training accuracy increased slightly from 28.77% to 32.63%, while evaluation accuracy remained stable at around 31.3%.

These results indicate that the model was learning effectively, with no obvious signs of overfitting. The accuracy of approximately 31% is considered reasonable for a 256-way classification task at the byte level.



Figure 4.8: A visualization of CPU usage across multiple cores during BLT training (green indicates usage, red indicates idle).

Text Generation Capabilities

To assess the model's text generation abilities, we provided it with various prompts. While the generated text showed some coherence, it still contained many nonsensical elements. For example:

Prompt: "The company announced"

Generated: "The company announceda beang anine s s u in n in Sain che
aint on ont t s r a s tore bor in ar a ing as f lan a mi int al"

This output suggests that while the model has learned some patterns in byte-level data, further refinement is needed to produce fully coherent text.

Implications for Technical Assistance System

The BLT experiment provided valuable insights into byte-level processing of text data. While the model showed promise in learning patterns and achieving reasonable accuracy, the current implementation may not be immediately suitable for deployment in our technical assistance system. However, the byte-level approach demonstrates potential for handling diverse data types and could be further explored for specific tasks within our system, particularly those involving low-level data processing or compression.

4.9 New Experiment

We start a new experiment. We take code T5 + model to analyse the logs from the new system we created with PonySDK. What we analysed is by creating a script to track how many objects are detected correctly. We observed that it would be a great idea to track the items using dictionary.

Further, we believe our hypothesis that the transformer model will work. So there are many use cases for the Machine learning. We found that 18/85 widgets had 80% or more and 9/85 had 90% correctly being generated using the transformer model code T5+. In the future, we will need more script to complete it. What we want to do in the future? We rewrote the script in java, as our code base will be working with Cockpit, the java based system that is essential for working of the code system.

We discuss using CodeBleu and other metrics, but finally we decided to go with a simpler statistical tool that can calculate how 2 metrics are similar.

4.10 Future work

By implementing these optimizations, we expect to achieve good prediction and a significant bandwidth reduction in WebSocket communications, enabling more efficient data transfer and lower latency. The dynamic dictionary model, combined with generative AI, will allow us to adapt to changing data patterns and achieve superior performance compared to static approaches.

Chapter 5

Conclusion

This research has elucidated significant conceptual advancements across multiple domains pertinent to WebSocket optimization in financial trading systems. Our investigation has established a fundamental paradigm shift from modality-specific artificial intelligence approaches toward universal byte-level methodologies capable of directly processing and predicting raw digital data. This paradigm transcends the constraints of traditional tokenization, as evidenced across byte-level transformer architectures, state space models, and latent space approaches. A consistent observation throughout our examination was the critical equilibrium between model architecture and computational efficiency—with transformer-based models demonstrating superior capacity for capturing complex dependencies while state space formulations provide enhanced scaling properties for extended sequences. The patch-based processing paradigm emerged as a particularly valuable architectural innovation, effectively balancing granular pattern recognition with computational tractability.

The integration of generative artificial intelligence with established information theory concepts has demonstrated considerable promise. Lightweight dictionary-based approaches [10] achieve remarkable compression efficiency without the computational overhead of deep neural architectures, while code-specialized models such as CodeT5 [7] illustrate that domain-specific structural awareness significantly enhances prediction accuracy for structured data streams. This hybrid approach—combining traditional compression techniques with targeted machine learning—provides a pragmatic equilibrium particularly suited to latency-constrained environments such as financial trading platforms.

Our examination of WebSocket optimization has revealed that the most efficacious approach is not a singular technology but rather a methodical orchestration of complementary techniques, with dynamic dictionaries discovered during runtime rather than statically defined. The directed graph conceptualization of data flows enables sophisticated pattern recognition across transaction boundaries, addressing a fundamental limitation in current methodologies. Although computational complexity remains a significant challenge, particularly for client-side implementations, the evolving efficiency of byte-level models suggests an imminent advancement in practical deployment.

As artificial intelligence continues to transform network optimization, these findings indicate an emerging communication paradigm in which predictive intelligence becomes an integral component of data exchange protocols. This vision extends beyond incremental optimization of existing technologies to fundamentally redefine communication as a collaborative predictive process between endpoints, a transformation that promises to revolutionize not only financial systems but distributed computing architectures across domains where latency and bandwidth constraints have historically limited innovation.

By predicting data before it's needed, we can create more responsive, efficient applications that provide better user experiences while reducing bandwidth requirements. As these technologies mature, we anticipate their widespread adoption across various domains, from financial trading platforms to multimedia applications and beyond.

Chapter 6

References

6.1 References

- [1] Zheng, Yao, Liang, Weizhi, Zhao, Denny, Jiang, Zhangyang, Peng, Baolin, Dong, Li, Gu, Jiatao, Shou, Linjun, Zeng, Michael, and Wei, Furu. “ByteGPT: Beyond Language Models for Byte-Level Digital World Simulation.” *arXiv preprint arXiv:2402.19155*, 2024. <https://arxiv.org/pdf/2402.19155>. 1, 5, 13
- [2] Yu, Lili, Simig, Daniel, Gu, Colin, Hayase, Jose, Kusupati, Aditya, Touvron, Hugo, Ott, Myle, Xiong, Wenhan, Majumder, Bodhisattwa Prasad, Lewis, Mike, and Zettlemoyer, Luke. “MegaByte: Predicting Million-byte Sequences with Multiscale Transformers.” *arXiv preprint arXiv:2305.07185*, 2023. <https://arxiv.org/pdf/2305.07185>. 7
- [3] Dao, Tri, Gu, Albert, Shen, Kaitlyn, Arora, Stefano, Karamcheti, Siddharth, Liang, Percy, and Ré, Christopher. “MambaByte: Token-Free State Space Models for Byte-Level Language Modeling.” *arXiv preprint arXiv:2401.13660*, 2024. <https://arxiv.org/pdf/2401.13660>. 1, 9
- [4] Meta AI Research. “Byte Latent Transformer: Patches Scale Better than Tokens.” 2024. <https://ai.meta.com/research/publications/byte-latent-transformer-patches-scale-better-than-tokens/>. 1, 11
- [5] Puneet Mathur, Vlad I. Morariu, Aparna Garimella, Franck Dernoncourt, Jiuxiang Gu, Ramit Sawhney, Preslav Nakov, Dinesh Manocha, and Rajiv Jain, “DocScript: Document-level Script Event Prediction,” in *Proc. of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, Torino, Italia, May 2024, pp. 5140–5155. <https://aclanthology.org/2024.lrec-main.458/>. 14
- [6] Xinyue Cui and Swabha Swayamdipta, “Annotating FrameNet via Structure-Conditioned Language Generation,” *arXiv preprint arXiv:2406.04834*, 2024. <https://doi.org/10.48550/arXiv.2406.04834>. 14
- [7] Wang, Yue, Wang, Weishi, Joty, Shafiq, and Hoi, Steven C.H. “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation.” In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021. <https://aclanthology.org/2021.emnlp-main.685.pdf>. 1, 21, 37
- [8] Feng, Zhangyin, Guo, Daya, Tang, Duyu, Duan, Nan, Feng, Xiaocheng, Gong, Ming, Shou, Linjun, Qin, Bing, Liu, Ting, Jiang, Daxin, and others. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages.” In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.

- [9] Lu, Shuai, Guo, Daya, Ren, Shuo, Huang, Junjie, Svyatkovskiy, Alexey, Blanco, Ambrosio, Clement, Colin, Drain, Dawn, Jiang, Daxin, Tang, Duyu, and others. “CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation.” *arXiv preprint arXiv:2102.04664*, 2021. 21
- [10] Wan, L., Alpcan, T., Kuijper, M., & Viterbo, E. “Lightweight Conceptual Dictionary Learning for Text Classification Using Information Compression.” *arXiv preprint arXiv:2405.01584v1*, 2024. URL: <https://arxiv.org/abs/2405.01584>. 1, 14, 21, 37
- [11] Chaqfeh, M., Haseeb, M., Hashmi, W., Inshuti, P., Ramesh, M., Varvello, M., Zaffar, F., Subramanian, L., and Zaki, Y. “To Block or Not to Block: Accelerating Mobile Web Pages On-The-Fly Through JavaScript Classification.” NYUAD Capstone Project 2 Report, Spring 2023, Abu Dhabi, UAE. URL: <https://arxiv.org/pdf/2106.13764>. 1, 21
- [12] Sirbu, A.-G. “Deobfuscating JavaScript Code Using Character-Based Tokenization.” *STUDIA UNIV. BABES–BOLYAI, Informatica, LXVIII(2)*, 2023. DOI: 10.24193/subbi.2023.2.01. 22
- [13] Melicher, W., Fung, C., Bauer, L., & Jia, L. “Towards a Lightweight, Hybrid Approach for Detecting DOM XSS Vulnerabilities with Machine Learning.” In *Proceedings of the Web Conference 2021 (WWW ’21)*, 2021. DOI: 10.1145/3442381.3450062.
- [14] Zheng, J., Rezagholizadeh, M., & Passban, P. “Dynamic Position Encoding for Transformers.” (Preprint, 2024). URL: [URL as provided in the document].
- [15] Wang, X., Li, S., & Ji, H. “CODE4STRUCT: Code Generation for Few-Shot Event Structure Prediction.” University of Illinois Urbana-Champaign, 2023. URL: <https://github.com/xingyaoww/code4struct>. 14
- [16] Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer, 2020. URL: <https://web.stanford.edu/~hastie/ElemStatLearn/>.
- [17] Sutskever, I., Martens, J., and Hinton, G. “Generating Text with Recurrent Neural Networks.” In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011, pp. 1017–1024. 9
- [18] Wang, S., Li, B., Khabsa, M., Fang, H., and Ma, H. “Linformer: Self-Attention with Linear Complexity.” *arXiv preprint arXiv:2006.04768*, 2020. 9
- [19] Choromanski, K., Likhoshesterov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, L., Belanger, D., Colwell, L., and Weller, A. “Rethinking Attention with Performers.” In *International Conference on Learning Representations (ICLR)*, 2021. 9
- [20] Kitaev, N., Kaiser, Ł., and Levskaya, A. “Reformer: The Efficient Transformer.” In *International Conference on Learning Representations (ICLR)*, 2020. 9
- [21] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., and Polosukhin, I. “Attention is All You Need.” In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [22] Child, R., Gray, S., Radford, A., and Sutskever, I. “Generating Long Sequences with Sparse Transformers.” *arXiv preprint arXiv:1904.10509*, 2019.
- [23] Beltagy, I., Peters, M.E., and Cohan, A. “Longformer: The Long-Document Transformer.” *arXiv preprint arXiv:2004.05150*, 2020.

- [24] Sennrich, R., Haddow, B., and Birch, A. “Neural Machine Translation of Rare Words with Subword Units.” In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1715–1725.
- [25] Wu, Y., Schuster, M., Chen, Z., Le, Q.V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., and others. “Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation.” *arXiv preprint arXiv:1609.08144*, 2016.
- [26] Al-Rfou, R., Choe, D., Constant, N., Guo, M., and Jones, L. “Character-Level Language Modeling with Deeper Self-Attention.” In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019, vol. 33, pp. 3159–3166.
- [27] Liu, P.J., Saleh, M., Pot, E., Goodrich, B., Sepassi, R., Kaiser, L., and Shazeer, N. “Generating Wikipedia by Summarizing Long Sequences.” In *International Conference on Learning Representations (ICLR)*, 2018.
- [28] Dong, L., Xu, S., and Xu, B. “Speech-Transformer: A No-Recurrence Sequence-to-Sequence Model for Speech Recognition.” In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 5884–5888.
- [29] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.” In *International Conference on Learning Representations (ICLR)*, 2021.

Chapter 7

Acknowledgement

I would like to express my sincere gratitude to Professor Youssef Mesri, the Director of Masters at Mines Paris - PSL, for providing me with the invaluable opportunity to pursue my academic aspirations in this prestigious program. His leadership and commitment to excellence have significantly shaped my educational journey. I am equally indebted to Mathieu for recognizing my potential and selecting me to collaborate with Smart Trade, an experience that has profoundly enriched my understanding of cutting-edge financial technologies.

My deepest appreciation extends to all the faculty members at Mines Paris - PSL who have generously shared their knowledge and expertise throughout my studies. Their guidance and constructive feedback have been instrumental in refining my research methodology and analytical skills. I would also like to acknowledge my colleagues and peers whose intellectual discussions and collaborative spirit created a stimulating environment conducive to innovation and discovery.

I also acknowledge the utility of various productivity tools that facilitated the preparation of this document, including Visual Studio Code for development, Lucidchart and Mermaid for visualization, Grammarly for editorial refinement, and reference management tools such as Google Scholar and Perplexity. Assistance tools like ChatGPT and Google Translate helped streamline the formatting and organization of technical content into academic prose.

Finally, I am grateful to my family and friends for their unwavering support and encouragement throughout this challenging yet rewarding academic pursuit. Their belief in my abilities has been a constant source of motivation, especially during difficult times.